TEACHME-LAKEHOUSE · TRAINING PROGRAMME

Module 1 — Foundations

What all these words mean, and what we actually built with them

01

16 HOURS | 22 LESSONS | 10 APPLICATION DEVELOPERS, ZERO DATA-ENGINEERING BACKGROUND

AUTHOR
Surender Sara

COMPANY
Northbay Solutions

ISSUED
August 2026

MODULE 1 · LESSON 01

Why Your App Database Can't Do This

You have all written SELECT * FROM orders WHERE id = ?. Today we ask for every order ever, grouped six ways.

1 · TWO DIFFERENT JOBS

OLTP

On-Line Transaction Processing

- One row at a time, by key
- Index seek — answer in 1 ms
- Thousands of small writes/sec
- Stores each ROW whole, together

OLAP

On-Line Analytical Processing

- Every row, then summarised
- Full scan — seconds, not ms
- Few huge reads, almost no writes
- Stores each COLUMN together

2 · WHAT THE QUERY MUST READ

```
SELECT MATKL, COUNT(*) FROM sap.s611 GROUP BY MATKL
```

ROW STORE — must read all 29 columns — 638,035,208 rows

COLUMN STORE — reads 1 of 29 — same answer

Same answer. 97% less data off the disk.

3 · OUR REAL VOLUMES — AND WHY WE COPY THEM OUT

ZHOCIDC

1,377,080,716

rows · POS basket lines

`specs/download/sap_zhocidc.yaml`

S611

638,035,208

rows · scan / GP base

`specs/download/sap_s611.yaml`

S603

648,802,247

rows · distress base

`specs/download/sap_s603.yaml`

WHY NOT JUST ASK SAP?

SAP runs the tills. Its CPU is not spare capacity for your GROUP BY.

So we copy it out — date-windowed, 8 parallel lanes, capped on purpose.

`docs/handoff/SAP-BASE-TABLES.md`

IN PLAIN ENGLISH

OLTP is a filing clerk: fetch file 4471, now. OLAP is an auditor: bring every file since 2018, grouped by branch.

MODULE 1 · LESSON 01

L01 · Why Your App Database Can't Do This

The point

Your app's database is built to fetch one row fast; analytics asks for every row at once, and that is a different machine.

Key ideas

- **OLTP** (your app DB) = many tiny reads/writes by key, answered in milliseconds via an index seek.
- **OLAP** (analytics) = few enormous reads, no writes, answered in seconds by scanning everything.
- A **row store** keeps each row's fields together, so reading one column still drags all 29 off the disk.
- A **column store** keeps each column together, so `SELECT MATKL, COUNT(*) … GROUP BY MATKL` reads 1 of 29 columns — about **97% less data**.
- Scale is the whole argument: `ZHOCIDC` 1,377,080,716 rows, `S603` 648,802,247, `S611` 638,035,208.
- You never report off production: SAP's CPU runs the tills, and one analyst's `GROUP BY` can stall a checkout lane.
- So we copy data out instead — date-windowed, capped at 8 parallel JDBC lanes so we never overwhelm the source.

Words you'll hear

TERM	MEANS
OLTP	transaction workload — one row, by key, right now
OLAP	analytical workload — all rows, summarised
Row store	fields of one row stored next to each other
Column store	values of one column stored next to each other
Full scan	reading every row because there is no useful index
Watermark	the column we use to pull only new rows (`SPTAG` on S611)

In this repo

- `src/glue/specs/download/sap_s611.yaml` — 638 M rows: `watermark_column: SPTAG`, `jdbc_hash_partitions: "8"`, and the comment explaining why the lane count is capped.
- `src/glue/specs/download/sap_zhocidc.yaml` — the 1.38 B-row POS table, pulled date-windowed, never full.
- `src/glue/specs/download/sap_s603.yaml` — the third giant.
- `docs/handoff/SAP-BASE-TABLES.md` — the real row counts and the rule "Giants: pull date-windowed, never full".

Do this

Open `src/glue/specs/download/sap_s611.yaml`. Count the columns in `schema:`, then work out how many bytes a row-store scan reads versus a column store that needs only `MATKL`.

You've got it when you can…

Explain why `SELECT id FROM orders WHERE id = 42` and `SELECT store, SUM(sales) FROM orders GROUP BY store` want opposite storage layouts — and why we never run the second one against SAP.

MODULE 1 · LESSON 02

Lake vs Warehouse vs Lakehouse

Cheap and dumb, fast and strict, or both? What each one is good at — and which one we built.

1 · DATA LAKE

DATA LAKE

a folder of files on object storage

COST	Lowest — S3 per GB
FLEXIBILITY	Any file, any shape
SPEED	Slow — scan whole files
SCHEMA	None until you read it
GOVERNANCE	Bucket policies only

GOOD AT

Keeping everything, cheaply, forever

BAD AT

Answering a question quickly

`glue_engine/writers/raw_landing.py`

2 · DATA WAREHOUSE

DATA WAREHOUSE

a locked, indexed, tuned database

COST	Highest — storage + compute
FLEXIBILITY	Tables only — load it first
SPEED	Fastest — it owns the bytes
SCHEMA	Strict, enforced on write
GOVERNANCE	Rich — roles and grants

GOOD AT

Fast, governed answers at scale

BAD AT

Cost — and data it has not loaded

`src/dbt/models/marts/gold/`

3 · LAKEHOUSE — WHAT WE BUILT

LAKEHOUSE

the folder that acts like a database

COST	Lake price for storage
FLEXIBILITY	Open files, table rules
SPEED	Fast where it has to be
SCHEMA	Enforced — and evolvable
GOVERNANCE	Catalog + Lake Formation

GOOD AT

Both — open files, warehouse reads

BAD AT

More moving parts to operate

`glue_engine/writers/s3_tables.py`

WHAT WE ACTUALLY BUILT — THE THREE PIECES

S3 Tables — lake economics

`infra/modules/s3-data-lake/main.tf`

Redshift Serverless — warehouse speed

`infra/modules/redshift-serverless/`

Iceberg — the bridge between them

`infra/modules/catalog_federation/`

IN PLAIN ENGLISH

Lake = a folder of files. Warehouse = a locked, indexed database. Lakehouse = the folder that behaves like the database.

MODULE 1 · LESSON 02

L02 · Lake vs Warehouse vs Lakehouse

The point

A lake is cheap and dumb, a warehouse is fast and strict, and a lakehouse is the folder of files that has been taught to behave like a database — which is what we built.

Key ideas

- **Data lake** = files on object storage. Cheapest, accepts anything, no schema until something reads it — and slow, because answering anything means scanning whole files.
- **Data warehouse** = a database that owns its bytes. Fastest and best governed, but you pay for storage *and* compute, and it can only answer questions about data you loaded into it first.
- **Lakehouse** = open files in the lake + a table layer on top + engines that can read them. Lake storage price, warehouse-shaped reads.
- The trade-off axes worth memorising: **cost · flexibility · speed · schema · governance**.
- We use all three ideas at once: **S3 Tables** for lake economics, **Redshift Serverless** for warehouse speed, **Iceberg** as the bridge that lets Redshift read files it does not own.
- Bronze and Silver stay open files on S3 Tables; Gold is materialised as native Redshift tables because that last hop is where speed is bought.
- The cost of a lakehouse is operational: more moving parts (catalog, table format, grants) than a single database.

Words you'll hear

TERM	MEANS
Object storage	S3 — cheap, durable, dumb; you PUT and GET whole files
Table format	the metadata layer that makes a folder of files act like a table (Lesson 06/07)
Catalog	the directory that tells engines which tables exist and where
MPP	massively parallel processing — many nodes answering one query
Open format	data readable by any engine, not locked inside one vendor

In this repo

- `infra/modules/s3-data-lake/main.tf` — the Bronze and Silver **S3 Tables** buckets: lake economics with AWS-managed Iceberg maintenance.
- `infra/modules/redshift-serverless/` — the warehouse that serves Gold and Power BI.
- `infra/modules/catalog_federation/` — the federated Glue catalog that lets Redshift see lake tables.
- `src/glue/glue_engine/writers/s3_tables.py` — where we actually write lakehouse tables.
- `src/dbt/models/marts/gold/` — the part that lives *inside* the warehouse.

Do this

Open `infra/modules/s3-data-lake/main.tf` and `infra/modules/redshift-serverless/main.tf` side by side, and say out loud which layer of our pipeline each one stores.

L02 · Lake vs Warehouse vs Lakehouse

You've got it when you can...

Name one question you would answer from S3 Tables and one you would answer from Redshift — and say why putting either in the other place would cost more or run slower.

MODULE 1 · LESSON 03

The Four Layers — and What Changes at Each

Raw → Bronze → Silver → Gold. Every hop has exactly one job. Never do two jobs in one hop.

RAW

the untouched dump

- Byte-for-byte from source
- Parquet files on S3, per cycle
- Never edited. Never deleted.
- 7-year retention

WHAT CHANGES
Nothing. That's the point.

`jobs/source_download.py`
`writers/raw_landing.py`

Why: replay anything, anytime

BRONZE

typed & deduplicated

- Real data types applied
- Iceberg MERGE on the PK
- Audit cols: _run_id, _ingested_at
- Still 1:1 with the source

WHAT CHANGES
Shape, not meaning

`jobs/bronze_pull.py`
`specs/bronze/*.yaml`

Rule: NO business logic here

SILVER

cleansed & conformed

- Business rules applied
- Names/units standardised
- Source logic rebuilt (ABAP → Spark)
- Joins across tables allowed

WHAT CHANGES
Meaning is added

`jobs/bronze_to_silver.py`
`glue_engine/abap/`

Trap: joins can silently double rows

GOLD

shaped for the question

- Star schema: facts + dimensions
- Aggregated to the report grain
- Lives INSIDE Redshift (native)
- Built by dbt, tested by dbt

WHAT CHANGES
Answers, not records

`src/dbt/models/marts/gold/`
`unified_sales.sql`

Power BI reads only this

ONE ROW, ALL FOUR LAYERS

`sap.zsdcc dump → bronze.sap.zsdcc (typed, merged) → silver.sap.zsdcc (dept text joined) → gold.unified_sales (site × date × dept × scenario)`

IN PLAIN ENGLISH

Raw = the photo of the receipt · Bronze = typed into the system · Silver = cleaned and standardised · Gold = the line on the report

MODULE 1 · LESSON 03

L03 · The Four Layers — and What Changes at Each

The point

Data moves through four layers, and **each hop has exactly one job**. Doing two jobs in one hop is how pipelines become unmaintainable.

Key ideas

- **Raw** — byte-for-byte from the source. Never edited, never deleted. It exists so you can *replay* anything.
- **Bronze** — the same data, typed and deduplicated. Iceberg `MERGE` on the primary key. **No business logic.**
- **Silver** — cleansed and conformed. Business rules live here; source logic (SAP ABAP) is rebuilt in PySpark.
- **Gold** — star schema, aggregated to the grain the report asks for. Lives *inside* Redshift as native tables.
- **The discipline:** if you can't say which layer a transformation belongs in, you don't understand it yet.
- Layers are **cheap** (S3) until Gold, which is **fast** (Redshift). That trade is deliberate.
- Every layer is reproducible from the one before it — so a bug is fixed by reprocessing, not by patching data.

Words you'll hear

TERM	MEANS
Medallion	The raw→bronze→silver→gold layering pattern
Conform	Make different sources agree on names, units and keys
Grain	What one row represents (e.g. *one site, one day, one dept*)
Star schema	One fact table surrounded by dimension tables
Replay	Rebuild a layer from the layer before it

In this repo

- `src/glue/glue_engine/jobs/source_download.py` — writes **Raw**
- `src/glue/glue_engine/jobs/bronze_pull.py` — Raw → **Bronze**
- `src/glue/glue_engine/jobs/bronze_to_silver.py` + `glue_engine/abap/` — Bronze → **Silver**
- `src/dbt/models/marts/gold/unified_sales.sql` — Silver → **Gold**

Do this

Open `bronze_pull.py` and `unified_sales.sql` side by side. List three things Gold does that Bronze deliberately does *not*.

You've got it when you can…

Take any transformation — "convert SAR to USD", "drop test stores", "add a department name" — and say which layer it belongs in, and why.

MODULE 1 · LESSON 04

The Whole Platform on One Slide

Come back to this map for the next 40 hours. Every other lesson zooms into one box here.

THE DATA PATH

WHAT MAKES IT RUN

ORCHESTRATION

EventBridge cron → dispatcher Lambda
→ Step Functions → Glue jobs

Barriers gate every phase hand-off

`src/lambdas/*_barrier/`

CONTROL PLANE

DynamoDB: runs · watermarks
pipeline-state · lineage_edges

Knows what ran, when, how long

`src/shared/control_plane/`

CATALOG & GOVERNANCE

Glue federated catalog
Lake Formation tag-based grants

6 KMS keys · one per layer

`modules/catalog_federation/`

BUILD & RUN

Terraform IaC · 30 modules
Bitbucket Pipelines + AWS OIDC

Dev · QA · Prod — separate deploys

`infra/` · `bitbucket-pipelines.yml`

IN PLAIN ENGLISH

Top row = the assembly line (data moves left to right). Bottom row = the factory itself — the scheduler, the logbook, the locks and the build system.

L04 · The Whole Platform on One Slide

The point

One map you return to for the next 40 hours. **Every other lesson is a zoom into one box on this slide.**

Key ideas

- **Top row = the assembly line.** Sources → Raw → Bronze → Silver → Gold → Power BI. Data only moves left to right.
- **Bottom row = the factory.** Orchestration, control plane, catalog/governance, and build/deploy. None of it touches the data — it *runs* the data.
- **P1 / P2 split:** one job pulls from the source (P1), another loads Bronze (P2). After P1, we never hit SAP again — that's deliberate.
- **Two storage worlds:** Bronze/Silver are Iceberg on S3 Tables (cheap, open). Gold is native Redshift (fast, closed). dbt is the bridge.
- **Barriers** gate each phase hand-off — nothing starts until the previous phase is provably complete.
- **The control plane knows everything:** what ran, when, how long, how many rows, and what came from what.
- **Three environments** (Dev/QA/Prod) deploy independently through the same pipeline.

Words you'll hear

TERM	MEANS
P1 / P2	Source-download job / Bronze-load job
Barrier	A gate that waits for a whole phase to finish
Control plane	The bookkeeping tables (runs, watermarks, lineage)
Cycle	One daily end-to-end run, identified by date
Dispatcher	The Lambda that decides what runs today

In this repo

- `src/glue/glue_engine/jobs/` — the four job types on the top row
- `src/lambdas/dispatcher/` , `src/lambdas/*_barrier/` — orchestration
- `src/shared/shared/control_plane/` — runs · watermarks · pipeline-state · lineage
- `infra/` + `bitbucket-pipelines.yml` — how it all gets deployed

Do this

Point at any box on the slide and find the folder that implements it. All eight are findable in under a minute.

You've got it when you can…

Draw this map from memory on a whiteboard, and explain what would break if you deleted the **Raw** layer.

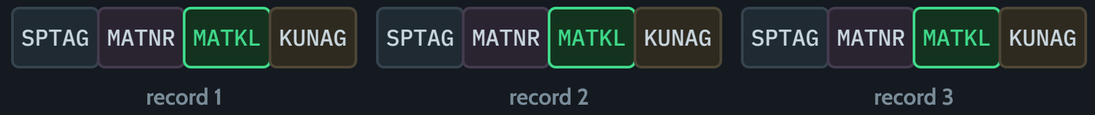
MODULE 1 · LESSON 05

Why Parquet, Not CSV

Same rows, different byte order — and that one choice decides whether 638 M rows is a query or a coffee break.

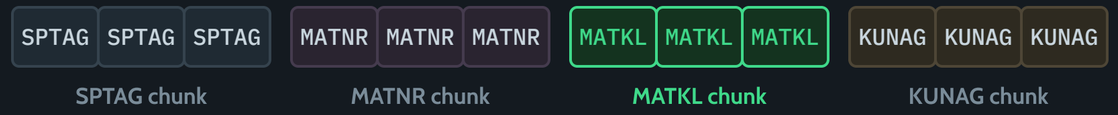
1 · HOW THE BYTES ACTUALLY SIT

CSV — one whole row, then the next row



read all 29

PARQUET — one whole column, then the next



read 1 chunk

2 · WHAT COLUMNAR BUYS YOU

COMPRESSION
Same-type values sit together — they squash hard

COLUMN PRUNING
Need 1 column of 29? Read 1 column of 29.

PREDICATE PUSHDOWN
Each chunk carries min/max — skip it unopened

3 · THE SAME 638 M ROWS, BOTH WAYS

CSV — not used here

Layout	row after row, interleaved
Types	none — every field is text
One column	still reads the whole file
Compressed	gzip: one lump, unsplittable

PARQUET — what we write

Layout	column chunk, then the next
Types	typed, encoded, dictionary
One column	reads 1 chunk of 29
Compressed	snappy, per chunk, splittable

WHY IT IS NOT OPTIONAL

S611 638,035,208 rows × 29 columns
S603 648,802,247 rows
ZHOCIDC 1,377,080,716 rows

In CSV, this pipeline never finishes.

`glue_engine/writers/raw_landing.py:227`

IN PLAIN ENGLISH

CSV is handwritten lines in a notebook. Parquet is a spreadsheet sorted into columns — so reading one column is cheap.

L05 · Why Parquet, Not CSV

The point

Same rows, different byte order: CSV writes a whole row then the next row, Parquet writes a whole column then the next — and that single choice is what makes 638 M rows queryable.

Key ideas

- **Row layout (CSV):** `SPTAG, MATNR, MATKL, KUNAG` for record 1, then all four again for record 2. Columns are interleaved, so you cannot read one without reading all of them.
- **Column layout (Parquet):** every `SPTAG` value together, then every `MATNR`, then every `MATKL`. Each block is a **column chunk**.
- **Compression:** values of the same type and similar range sit next to each other, so dictionary + run-length encoding squash them far harder than mixed row text.
- **Column pruning:** need 1 column of 29? Read 1 column of 29. CSV always reads the whole file.
- **Predicate pushdown:** each chunk stores min/max (and row counts) in its footer, so a `WHERE SPTAG >= …` skips whole chunks without opening them.
- **Splittable:** snappy compresses per chunk, so Spark can hand different chunks to different workers. A gzipped CSV is one lump — one worker, no parallelism.
- CSV also has **no types**: every field is text, so every read pays a parse cost and every schema is a guess.
- At our volumes this is not a preference. `S611` 638,035,208 rows × 29 columns, `S603` 648,802,247, `ZHOCIDC` 1,377,080,716 — in CSV the pipeline never finishes.

Words you'll hear

TERM	MEANS
Columnar	values of one column stored contiguously
Column chunk	one column's values for one block of rows
Row group	a horizontal slice of the file, split into column chunks
Footer / metadata	per-chunk min/max, counts and offsets at the end of the file
Predicate pushdown	using that metadata to skip chunks before reading them
Splittable	a file many workers can read in parallel

In this repo

- `src/glue/glue_engine/writers/raw_landing.py:227` — `df.write.mode("overwrite").parquet(prefix)`; the writer refuses any format other than Parquet.
- `src/glue/specs/download/sap_s611.yaml` — `format: parquet` plus the typed `schema:` block; types are declared once, not re-guessed on every read.
- `src/glue/glue_engine/writers/s3_tables.py` — downstream Iceberg tables, also Parquet underneath.

Do this

Land any small table to raw, then list the S3 prefix. Note the `.snappy.parquet` part files and the `_SUCCESS` marker, and compare the total size against the same row count as CSV.

You've got it when you can…

Explain, without saying "it's faster", exactly which bytes a `GROUP BY MATKL` reads from a Parquet file and which bytes it reads from the same data as CSV.

MODULE 1 · LESSON 06

A Folder of Files Is Not a Table

Parquet on S3 takes an afternoon. Making it behave like a table is the whole problem.

1 · WHAT YOU HAVE

A PREFIX ON S3

```
sap/sap.s611/cycle=2026-08-10/
  part-00000-….parquet
  part-00001-….parquet
  part-00002-….parquet
  … 9 more files
  _SUCCESS
```

Nothing here says "table".

`glue_engine/writers/raw_landing.py`

2 · WHAT BREAKS

NO ATOMICITY ✗

A reader can arrive halfway through a write

Half the files is not half the truth

NO SCHEMA ✗

Types live inside each file, not the set

File 9 renames a column — who wins?

NO UPDATES ✗

S3 objects are immutable — there is no UPDATE

Change one row = rewrite the whole file

NO CONCURRENCY ✗

Two writers, no lock — last one wins

Readers watch the file list move

3 · THE MISSING LAYER — METADATA ABOUT THE FILES

DATA FILES

The same Parquet as before

Immutable — only added

Never edited in place

`cycle=<id>/part-*.parquet`

→

MANIFESTS

A list of which files

belong to the table

+ row counts, min/max

so files can be skipped

without being opened

→

SNAPSHOT

One pointer = the table

Swap it = atomic commit

Old pointer still reads

the old set of files

nobody sees half a write

WHAT THAT FIXES

- ✓ Atomic: all of a write, or none
- ✓ Schema travels with the table
- ✓ Row-level updates and deletes
- ✓ Many readers, one stable view

`glue_engine/writers/s3_tables.py`

IN PLAIN ENGLISH

A folder of files is a pile of receipts. A table format is the index card saying which receipts count — and as of when.

L06 · A Folder of Files Is Not a Table

The point

Putting Parquet on S3 is an afternoon's work; making that pile of files behave like a table — atomic, typed, updatable, safe for concurrent readers — is the problem a *table format* exists to solve.

Key ideas

- What you actually have is a **prefix**: a dozen `part-*.parquet` objects under `sap/sap.s611/cycle=.../`. Nothing in there says "table".
- **No atomicity.** A reader can arrive halfway through a write and see 6 of 12 files. Half the files is not half the truth — it is a wrong answer.
- **No schema.** Types live inside each file, not across the set. When file 9 renames or retypes a column, nothing decides who wins.
- **No updates.** S3 objects are immutable: there is no `UPDATE`. Changing one row means rewriting the file that holds it.
- **No concurrency.** Two writers, no lock, last one wins — and readers watch the file list move under them.
- The fix is a **metadata layer** that sits above the unchanged data files: **manifests** (which files belong to the table, with row counts and min/max) and a **snapshot** (one pointer that *is* the table).
- Swapping the snapshot pointer is a single atomic commit, so nobody ever sees half a write — and the old pointer still reads the old set of files.
- Our `_SUCCESS` marker in raw is the poor-man's version of this: data files written first, marker last, and readers refuse a prefix with no marker.

Words you'll hear

TERM	MEANS
Table format	the metadata spec that turns files into a table
Manifest	a list of data files plus per-file stats
Snapshot	an immutable pointer to one exact set of files = one table version
Commit	publishing a new snapshot, atomically
Atomicity	all of a write is visible, or none of it
`_SUCCESS`	marker file proving a write finished (our raw-zone version)

In this repo

- `src/glue/glue_engine/writers/raw_landing.py` — the folder-of-files problem in the flesh: per-cycle prefix, data files first, `_SUCCESS` last.
- `src/glue/glue_engine/sources/landed_files.py` — the reader that refuses a prefix without a `_SUCCESS` marker.
- `src/glue/glue_engine/writers/s3_tables.py` — where the real metadata layer takes over (Lesson 07).

Do this

Read the module docstring at the top of `writers/raw_landing.py` and explain why the `_SUCCESS` marker is written last — then list one failure it still does not protect against.

You've got it when you can…

Say what breaks when two jobs write to the same S3 prefix at the same time, and name the two metadata pieces that make it safe.

MODULE 1 · LESSON 07

Apache Iceberg: Snapshots, MERGE, Time Travel

Immutable files + one moving pointer. That single idea buys atomic writes, upserts, rollback and safe schema change.

1 · WHAT AN ICEBERG TABLE ACTUALLY IS

IMMUTABLE DATA FILES

Parquet files on S3

- Written once. Never edited.
- An UPDATE writes a NEW file
- and marks the old rows dead

You can't corrupt what you never edit

→

METADATA + MANIFESTS

The list of what's in the table

- Which files belong right now
- Row counts + min/max per file
- so a scan can skip whole files

This is what makes it a table

→

THE SNAPSHOT POINTER

One pointer → one metadata file

- A write adds new files, then
- MOVES the pointer — atomically
- Readers keep the old pointer

Rollback = move it back

2 · THE WRITES WE USE — src/glue/glue_engine/writers/s3_tables.py

`append(df)`

Add rows. Creates the table on the very first write. Used for CDC deltas.

`s3_tables.py:175`

`merge_into(df, key)`

Upsert on the natural key. Re-run the same input → the same row count. Idempotent.

`s3_tables.py:193`

`full_refresh(df)`

Replace every row, KEEP the table. One atomic overwrite, never a DROP.

`s3_tables.py:326`

`expire_snapshots()`

Drop snapshots older than 7 days, keep the last 100. Else metadata grows forever.

`s3_tables.py:407`

ICEBERG GIVES YOU THESE TOO — real syntax, not called anywhere in our repo yet

```
SELECT * FROM tbl VERSION AS OF <snapshot_id> · CALL cat.system.rollback_to_snapshot('ns.tbl', <id>)
```

IN PLAIN ENGLISH

Git for tables. Every write is a commit — new files, new manifest, then one pointer moves. Nothing is edited; nothing half-lands.

L07 · Apache Iceberg: Snapshots, MERGE, Time Travel

> Slide: `_render/L07-apache-iceberg.html`

The point

A folder of Parquet files can't be updated safely — two writers collide, a reader mid-scan sees half a write, and there is no `UPDATE`. Iceberg fixes all of that with one idea: **data files are immutable, and the "table" is just a pointer to a list of them.** Every write publishes a new list and moves the pointer in one atomic step. That single mechanism is where atomic writes, upserts, rollback and safe schema change all come from.

Key ideas

- **Data files are write-once.** An UPDATE never edits a Parquet file; it writes a new one and marks the old rows deleted.
- **Metadata + manifests are the table.** They list which files are in the table *right now*, plus per-file row counts and min/max stats so a scan can skip whole files.
- **A snapshot is a pointer.** Commit = write new files → write new manifest → move one pointer. Nothing is ever half-applied.
- **Readers hold the old pointer**, so a long query is never disturbed by a concurrent write. No locks, no dirty reads.
- **`MERGE INTO` is the upsert.** Re-running the same input produces the *same* row count — replays and Glue auto-retries stop being dangerous.
- **Rollback and time travel are free**, because the old snapshots still exist. You point at an older one.
- **Snapshots must be expired.** Every commit adds one; unbounded metadata slows query planning. Data-file compaction is a separate concern (S3 Tables does it for us — see L08).

Words you'll hear

TERM	MEANS
Snapshot	One immutable version of the whole table — a pointer to one metadata file
Manifest	The list of data files (+ stats) that make up a snapshot
Commit	Publishing a new snapshot by atomically moving the pointer
`MERGE INTO`	Upsert: update matched rows on a key, insert the rest
Merge key	The natural/primary key the MERGE matches on
Idempotent	Running it twice gives the same result as running it once
Time travel	Reading the table as of an older snapshot
Expire snapshots	Deleting old snapshots so metadata stops growing

L07 · Apache Iceberg: Snapshots, MERGE, Time Travel

In this repo

PATH	WHAT IT SHOWS
`src/glue/glue_engine/writers/s3_tables.py:175`	`append` — adds rows; falls back to CREATE on first write
`src/glue/glue_engine/writers/s3_tables.py:193`	`merge_into` — the real `MERGE INTO … WHEN MATCHED THEN UPDATE SET * …`, plus the CDC delete arm
`src/glue/glue_engine/writers/s3_tables.py:326`	`full_refresh` — replaces every row via one atomic `overwrite(lit(True))`, never a DROP
`src/glue/glue_engine/writers/s3_tables.py:407`	`expire_snapshots` — `CALL … system.expire_snapshots(older_than, retain_last=100)`
`src/glue/glue_engine/writers/s3_tables.py:539`	`_dedupe_latest` — collapses duplicate keys so MERGE can't hit a cardinality violation
`src/glue/glue_engine/jobs/bronze_pull.py:298-314`	The caller: specs that declare a `merge_key` upsert; everything else appends

The war story to tell: `bronze.sap.zncr01` held **446,611** rows against **438,645** at source. Cause: `append` on a full-snapshot re-pull. Fix: `merge_into` on the natural key. That is the entire business case for Iceberg in one number.

Do this

Open `writers/s3_tables.py` and read `merge_into` (line 193) top to bottom. Then answer, without running anything: **what happens if the same Glue job runs three times in a row on identical source data — with `append`, and with `merge_into`?** Now find the branch in `bronze_pull.py` that decides between them, and say what in the YAML spec flips it.

You've got it when you can…

Explain why a reader that started a query *before* a write, and finished *after* it, never sees a partially-written table — using only the words *files*, *manifest* and *pointer*.

MODULE 1 · LESSON 08

Amazon S3 Tables: Managed Iceberg

Same Iceberg format — AWS owns the bucket and does the housekeeping. What that trade buys, and what it costs.

1 · THE SHAPE — infra/modules/s3-data-lake/main.tf

TABLE BUCKET

An Iceberg-native bucket

- Two of them: bronze and silver
- AWS-managed compaction inside
- Encrypted with our own KMS key

`s3-data-lake/main.tf:11 · :30`

File cleanup: 7d bronze · 30d silver

NAMESPACE

A schema inside the bucket

- sap · ncr · ecommerce
- Same three in both buckets
- Made by a Terraform for_each

`main.tf:52 (bronze) · :59 (silver)`

bronze.sap.zsdcc → ns=sap, tbl=zsdcc

TABLE

NOT created by Terraform

- Tables register themselves on the first write from a Glue job
- Adding a table = one new YAML spec, not a terraform apply

`writers/s3_tables.py:143 _first_write_create`

2 · WHAT AWS TAKES OVER — AND WHAT YOU GIVE UP

AWS RUNS THIS FOR YOU

- Compaction — small files merged
- Unreferenced-file removal
- The physical bucket itself
- Encryption at rest, via KMS

`s3-data-lake/main.tf:14-27`

YOU STILL OWN THIS

- The writes: MERGE, full refresh
- Snapshot EXPIRY — metadata
- Partitioning, schema, merge keys
- Getting Spark's catalog conf right

`writers/s3_tables.py:407`

WHAT YOU GIVE UP

- aws s3 ls on that bucket name gives NoSuchBucket — no direct file access
- AWS-only: Iceberg is portable, this managed layer is not

`writers/s3_tables.py:60-67`

IN PLAIN ENGLISH

Self-managed Iceberg is running your own DB server. S3 Tables is the managed instance — same engine, someone else does the chores.

L08 · Amazon S3 Tables: Managed Iceberg

> Slide: `_render/L08-s3-tables.html`

The point

Iceberg (L07) gives you the format. Running it yourself means you also own the chores: compacting the thousands of small files a streaming/CDC pipeline produces, removing files no snapshot references any more, and babysitting the bucket. **S3 Tables is Iceberg as a managed service** — AWS runs the maintenance and owns the physical storage. You keep the writes, the keys and the schema. This lesson is about drawing that line precisely, because the parts AWS *doesn't* cover are the parts that bite.

Key ideas

- **Table bucket ≠ S3 bucket.** A table bucket is an Iceberg-native surface. `aws s3 ls` on its name returns `NoSuchBucket` — you cannot touch the files directly.
- **Three levels: table bucket › namespace › table.** We have two buckets (bronze, silver), each with the same namespaces (`sap` , `ncr` , `ecommerce`).
- **Terraform creates buckets and namespaces only.** Tables are *not* declared in infrastructure.
- **Tables register themselves on first write** from a Glue job. Adding a table is a new YAML spec, not a `terraform apply` .
- **AWS takes over:** data-file compaction, unreferenced-file removal, the physical bucket, encryption at rest with our KMS key.
- **You still own:** the writes, *snapshot* expiry (metadata, not data), partitioning, schema, merge keys — and getting the Spark catalog conf right, which is where the real pain lives.
- **What you give up:** direct file access, and portability. Iceberg runs anywhere; this managed layer is AWS-only.

Words you'll hear

TERM	MEANS
Table bucket	An Iceberg-native S3 bucket, addressed by ARN, not by object key
Namespace	A schema/grouping inside a table bucket (`sap` , `ncr` , `ecommerce`)
Compaction	Merging many small data files into fewer big ones so scans stay fast
Unreferenced file removal	Deleting files no live snapshot points at any more
Maintenance configuration	The Terraform block that tunes those retention windows
Managed physical bucket	The real storage AWS owns on your behalf; you never see it
Register on first write	The table is created by the job that first writes to it

L08 · Amazon S3 Tables: Managed Iceberg

In this repo

PATH	WHAT IT SHOWS
`infra/modules/s3-data-lake/main.tf:11`	`aws_s3tables_table_bucket.bronze`
`infra/modules/s3-data-lake/main.tf:30`	`aws_s3tables_table_bucket.silver`
`infra/modules/s3-data-lake/main.tf:14-22`	`maintenance_configuration` — unreferenced-file removal, 7 days (bronze) / 30 days (silver)
`infra/modules/s3-data-lake/main.tf:24-27`	`encryption_configuration` — `aws:kms` with the per-layer CMK
`infra/modules/s3-data-lake/main.tf:52` / `:59`	`aws_s3tables_namespace` for bronze / silver, `for_each = toset(var.namespaces)`
`src/glue/glue_engine/writers/s3_tables.py:143`	`_first_write_create` — the "tables register themselves" path
`src/glue/glue_engine/writers/s3_tables.py:60-67`	The `NoSuchBucket` discovery, and why the Spark catalog had to move to the Iceberg REST endpoint
`src/glue/glue_engine/writers/s3_tables.py:407`	`expire_snapshots` — the maintenance AWS does **not** do for you (see the ADR-0024 note)

Do this

Read the module header comment in `infra/modules/s3-data-lake/main.tf` (lines 1–9), then list every AWS object that exists for `bronze.sap.zsdcc`. Which of them appear in Terraform state, and which are created at runtime? Now find the retention difference between bronze and silver and argue why silver keeps files four times longer.

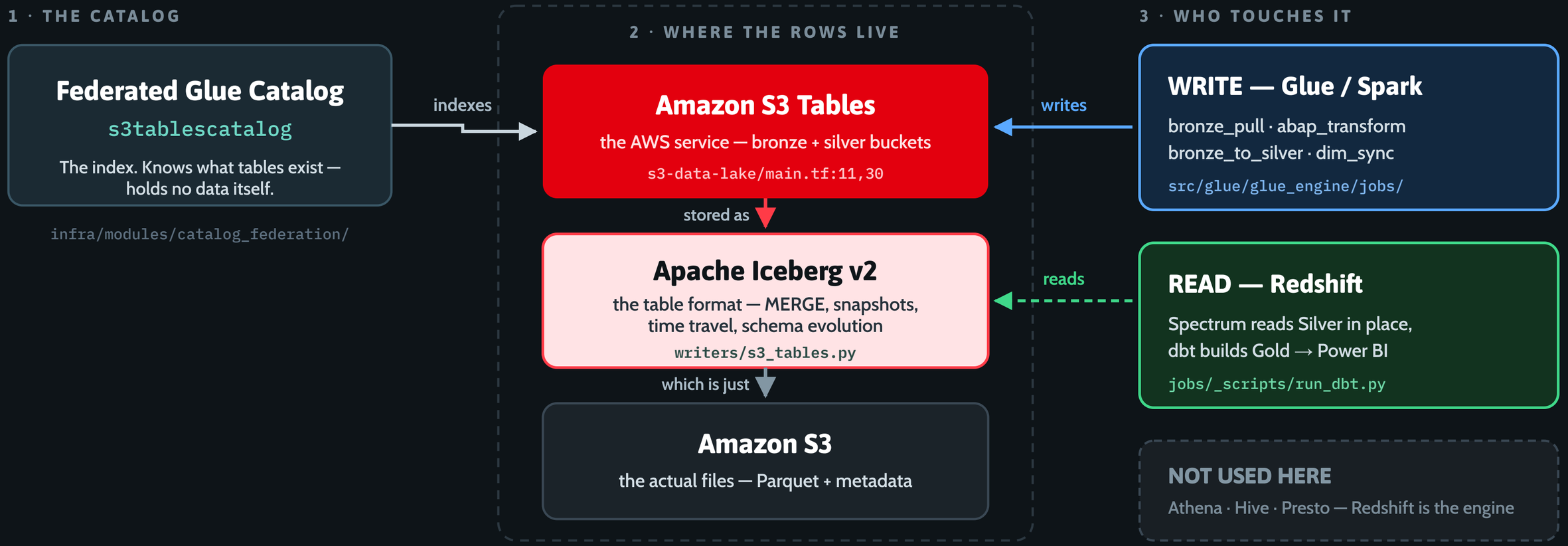
You've got it when you can…

Name one maintenance task S3 Tables performs for us and one it does not — and say which file in this repo would have to run for the second one.

MODULE 1 · LESSON 09

Where the Data Actually Lives

Catalog vs table format vs storage — three different things people all call “the database”



IN PLAIN ENGLISH

The **catalog** is the library index · **Iceberg** is the file format (like .docx) · **S3** is the hard drive. S3 Tables = AWS running Iceberg for you.

MODULE 1 · LESSON 09

L09 · Where the Data Actually Lives

The point

Three different things get called "the database". They are **not** the same, and mixing them up is the most common beginner error.

Key ideas

- **The catalog** (`s3tablescatalog`) is an *index*. It knows which tables exist and where — it holds **no data**.
- **The table format** (Apache Iceberg) is the *rulebook* that turns a pile of Parquet files into something with schema, atomic writes and updates.
- **The storage** (S3) is just *files*. Cheap, dumb, durable.
- **S3 Tables** = AWS running Iceberg for you — it manages compaction and maintenance on the physical bucket.
- **Writers** (Glue/Spark) and **readers** (Redshift) both go through the catalog to find the table, then touch the data directly.
- We use **no Glue Crawlers** — the federated catalog auto-mounts the S3 Tables buckets.
- Athena, Hive and Presto are **not used here**. Redshift is the query engine.

Words you'll hear

TERM	MEANS
Catalog	The index of tables (Glue Data Catalog)
Federated catalog	A catalog that mounts another system's tables
Table format	The metadata rules over the files (Iceberg)
Manifest / snapshot	Iceberg's list of files and point-in-time version
Managed bucket	The physical S3 bucket AWS owns for S3 Tables

In this repo

- `infra/modules/catalog_federation/` — the federated Glue catalog + Lake Formation grants
- `infra/modules/s3-data-lake/main.tf:11,30` — the bronze + silver table buckets
- `src/glue/glue_engine/writers/s3_tables.py` — the Iceberg read/write layer
- `infra/modules/s3/main.tf` — the plain S3 buckets (raw, artifacts, audit)

Do this

In `s3_tables.py` , find where a table's fully-qualified name is built. Explain which part is the catalog, which is the namespace and which is the table.

You've got it when you can…

Explain, without using the word "database", what happens when Redshift is asked for a Silver table it has never seen before.

MODULE 1 · LESSON 10

Spark & AWS Glue: Why Not Just a Python Script

One machine over 638M rows takes days; twelve take minutes. What Spark does with your code — and when it's overkill.

1 · THE SCALE PROBLEM

ONE PYTHON SCRIPT

pandas + one JDBC connection

- One CPU, one heap, one machine
- 638M rows never fit in RAM
- Fails at hour 6 → restart at 0

`sources/sap_hana.py:188-192`

VS

SPARK ON AWS GLUE

A cluster that lives 40 minutes

- 12 × G.2X workers, on demand
- The SAME PySpark code — Spark
- splits the work across them

`env/dev/glue_sources.tf:146-147`

WHAT WE ACTUALLY PULL

ZHOCIDC	1,377,080,716	POS line items
VBRP	684,592,844	billing items
S603	648,802,247	distress
S611	638,035,208	scan / GP

`specs/download/sap_s611.yaml:28 — 8 readers`

2 · THE FOUR IDEAS YOU MUST HOLD

PARTITIONS

One DataFrame = N chunks. Each chunk is one unit of work.

`specs/download/sap_s611.yaml:28`

EXECUTORS

12 worker JVMs chew chunks in parallel. More workers = faster.

`env/dev/glue_sources.tf:146`

LAZY EVALUATION

Nothing runs until you ask. Then each ask re-reads SAP — cache it.

`sources/sap_hana.py:1151-1160`

SHUFFLES

groupBy / join / dedupe moves rows between machines. Slow.

`writers/s3_tables.py:539`

WHEN SPARK IS THE WRONG TOOL

Under ~10M rows one machine wins — cluster startup and shuffle cost more than the work. Our dbt runner is a Glue Python-Shell job, not Spark.

IN PLAIN ENGLISH

Spark is a foreman, not a faster laptop. You describe the work; it splits the data across 12 machines and only starts when you ask.

MODULE 1 · LESSON 10

L10 · Spark & AWS Glue: Why Not Just a Python Script

> Slide: `_render/L10-spark-and-glue.html`

The point

You already know how to write `pandas.read_sql(...)`. That works right up to the moment the table is **638,035,208 rows** — then one CPU, one heap and one JDBC connection stop being an implementation detail and become the whole problem. Spark is not a faster laptop; it is a **foreman**. You describe the work, it splits the data into chunks, hands them to N machines, and doesn't start until you ask for a result. AWS Glue is that cluster rented for forty minutes and then thrown away.

Key ideas

- **Partitions** — a DataFrame is N chunks. A chunk is the unit of work, and it's the only reason parallelism is possible at all.
- **Executors** — worker JVMs that chew chunks concurrently. We run 12 × `G.2X` for the SAP download lanes. More workers, more chunks in flight.
- **Lazy evaluation** — nothing executes until an *action* (`count()`, `write`, `collect()`). Transformations only build a plan.
- **The lazy trap:** each action re-runs the plan. Without `.cache()`, `count()` + the watermark `agg(max)` + the write meant **3× the SAP load, 3× the cost**, and non-determinism if the source moved between passes.
- **Shuffles** — `groupBy`, `join`, `dropDuplicates`, window functions move rows *between machines*. That is the expensive operation; everything else is cheap by comparison.
- **Parallelism has a budget.** `jdbc_hash_partitions` is capped at 8 for the giants, not for Spark's sake, but because HANA + the transit gateway only tolerate so many concurrent connections.
- **When Spark is the WRONG tool:** under ~10M rows, cluster startup and shuffle cost more than the work. Our dbt runner is deliberately a Glue **Python-Shell** job, not Spark.

Words you'll hear

TERM	MEANS
Driver	The process running your `main()`; builds the plan, coordinates workers
Executor / worker	A JVM doing actual work on a slice of the data
Partition	One chunk of a DataFrame = one unit of parallel work
Transformation	Lazy — `select`, `filter`, `join`. Builds the plan, runs nothing
Action	Eager — `count`, `write`, `collect`. Triggers execution
Shuffle	Moving rows across the network so keys land together
`G.2X`	A Glue worker size (8 vCPU / 32 GB). `number_of_workers` sets how many
DPU-minute	What you are billed for: workers × how long they lived

L10 · Spark & AWS Glue: Why Not Just a Python Script

In this repo

PATH	WHAT IT SHOWS
`src/glue/glue_engine/`	The spec-driven engine: `sources/`, `transforms/`, `writers/`, `jobs/` — one engine, N YAML specs
`src/glue/glue_engine/jobs/bronze_pull.py`	A real Glue job end to end: resolve spec → read → PII redact → write → record the run
`src/glue/glue_engine/jobs/bronze_pull.py:255-256`	`# Count BEFORE write (DataFrame is lazy).` — laziness stated in the code
`src/glue/glue_engine/sources/sap_hana.py:1151-1160`	The `.cache()` comment: without it the JDBC pull re-executes once per action
`src/glue/glue_engine/sources/sap_hana.py:188-192`	The 5,000,000-row guard — a single-partition read of the giants OOMs on one executor
`src/glue/specs/download/sap_s611.yaml:28`	`jdbc_hash_partitions: "8"` — parallelism as configuration, with the reason in a comment
`infra/env/dev/glue_sources.tf:146-147`	`worker_type = "G.2X"`, `number_of_workers = 12` — the cluster, in Terraform

The scale, concretely: `ZHOCIDC` 1,377,080,716 · `VBRP` 684,592,844 · `S603` 648,802,247 · `S611` 638,035,208 rows.

Do this

Open `sources/sap_hana.py` and find `df.cache()` (line 1160). Read the comment above it, then trace which three actions would each have re-triggered the JDBC read. Now open `infra/env/dev/glue_sources.tf` and work out how many worker ENIs 3 lanes × 2 concurrent runs × 12 workers puts into one subnet — and why the comment says that number matters.

You've got it when you can…

Given a 4-million-row Oracle table that needs a daily join and aggregate, argue *for or against* Spark — and name the one number that decides it.

MODULE 1 · LESSON 11

Redshift Serverless: Where Answers Live

Massively parallel, columnar, rented by the second. And why Gold is copied into it instead of left on the lake.

1 · WHY IT ANSWERS FAST

MPP — MANY NODES, ONE QUERY

- The table is sliced across nodes
- Each node scans only its slice
- Leader merges the partial answers

`modules/redshift-serverless/main.tf:79`

COLUMNAR + ZONE MAPS

- Reads only the columns named
- Each column compressed apart
- Block min/max skips whole blocks

`marts/gold/unified_sales.sql`

RPUs — COMPUTE YOU RENT

- Nothing to size, patch or stop
- dev: 8 RPU base → 16 max
- prod: 32 RPU base → 64 max

`env/prod/terraform.tfvars:18-19`

2 · THE TWO KNOBS YOU CHOOSE — AND WHERE GOLD LIVES

DIST KEY

Decides which node a row lives on. dist='date' co-locates one day's rows on one slice.

`gold/unified_sales.sql:81`

SORT KEY

The physical order on disk. sort=['date','site'] lets a date filter skip whole blocks.

`gold/unified_sales.sql:82`

WHY GOLD LIVES INSIDE REDSHIFT

- Silver stays on the lake — Redshift reads it in place via Spectrum
- dbt writes Gold as NATIVE Redshift tables: local storage, dist + sort
- Power BI hits Gold only — no lake round-trip on every click

`jobs/_scripts/run_dbt.py:151-154 — CREATE EXTERNAL SCHEMA silver_external`

THE ACTUAL CONFIG — src/dbt/models/marts/gold/unified_sales.sql:76-83

```
{{ config(materialized='incremental', incremental_strategy='merge', unique_key=['site','date','aagm','scenario'], dist='date', sort=['date','site']) }}
```

IN PLAIN ENGLISH

The lake is the warehouse floor; Redshift is the shop front. dbt moves finished goods out front nightly — nobody waits.

L11 · Redshift Serverless: Where Answers Live

> Slide: `_render/L11-redshift-serverless.html`

The point

Everything up to Silver lives on the lake, and Redshift can read it there without copying anything. So why does **Gold get materialised inside Redshift**? Because Power BI asks the same question hundreds of times a day, and a query that scans someone else's files is never as fast as one that reads local, sorted, distribution-aware storage. Redshift is where the answers live; the lake is where the records live.

Key ideas

- **MPP (massively parallel processing).** The table is sliced across nodes; every node scans only its slice; the leader merges the partial answers. Add nodes, scan faster.
- **Columnar storage.** A query reads only the columns it names, each column is compressed on its own, and per-block min/max ("zone maps") let whole blocks be skipped unread.
- **Serverless = RPUs.** No cluster to size, patch or stop. You set a base and a ceiling and it scales with load: **dev 8 → 16 RPU, prod 32 → 64 RPU**.
- **Dist key** decides *which node* a row lives on. `dist='date'` co-locates a day's rows so date-grained joins don't move data across the network.
- **Sort key** decides the *physical order on disk*. `sort=['date','site']` is what makes a date-filtered dashboard query skip most of the table.
- **Silver is read in place** via Spectrum against the Glue catalog — nothing is copied in. That is L12/L13's territory.
- **Gold is native.** dbt writes real Redshift tables with dist + sort. Once a row reaches Gold, Spectrum is out of the picture — which is exactly why Gold is fast.
- **The namespace holds the data; the workgroup is the compute.** The namespace carries `prevent_destroy = true`; the workgroup is recreatable.

Words you'll hear

TERM	MEANS
MPP	Many nodes each scanning their own slice of one query
Columnar	Stored column-by-column, so reading 3 of 40 columns costs 3/40 of the I/O
Zone map	Per-block min/max stats that let the engine skip blocks entirely
RPU	Redshift Processing Unit — the serverless capacity unit you rent
Namespace	The logical database — holds the data, the schemas, the IAM roles
Workgroup	The compute + network surface: capacity, subnets, security groups, endpoint
Dist key	Column that decides which node/slice a row lands on
Sort key	Column order rows are physically stored in
Materialise	Write the query result out as a real table instead of re-computing it

L11 · Redshift Serverless: Where Answers Live

In this repo

PATH	WHAT IT SHOWS
`infra/modules/redshift-serverless/main.tf:51`	`aws_redshiftserverless_namespace` — the logical DB, IAM-only auth, KMS, `log_exports`
`infra/modules/redshift-serverless/main.tf:74-76`	`lifecycle { prevent_destroy = true }` — the namespace holds Gold; it must not be destroyable
`infra/modules/redshift-serverless/main.tf:79-84`	`aws_redshiftserverless_workgroup` — `base_capacity` / `max_capacity` in RPU
`infra/modules/redshift-serverless/main.tf:90`	`enhanced_vpc_routing = true` — all traffic stays in the VPC
`infra/env/dev/terraform.tfvars:19-20`	dev: base 8 RPU → max 16
`infra/env/prod/terraform.tfvars:18-19`	prod: base 32 RPU → max 64
`src/dbt/models/marts/gold/unified_sales.sql:76-83`	The config block: `materialized='incremental'`, `incremental_strategy='merge'`, `unique_key=['site','date','aagm','scenario']`, **`dist='date'`**, **`sort=['date','site']`**
`src/glue/glue_engine/jobs/_scripts/run_dbt.py:151-154`	`CREATE EXTERNAL SCHEMA silver_external … IAM_ROLE …` — the read path into Silver that feeds dbt

Do this

Open `src/dbt/models/marts/gold/unified_sales.sql` and read only the `config()` block (lines 76–83). The model's grain is site × date × dept × scenario. Explain why `dist='date'` and `sort=['date','site']` were chosen and not, say, `dist='site'` — then predict which Power BI page would get slower if you swapped them.

You've got it when you can…

Explain to a developer why we don't just point Power BI at the Silver Iceberg tables and skip Redshift entirely — in terms of what happens on the *hundredth* identical query of the day.

MODULE 1 · LESSON 12

Reading Data Redshift Doesn't Own

Redshift Spectrum — query files that live in S3 as if they were tables. Nothing is ever loaded in.

1 · TWO KINDS OF TABLE

NATIVE TABLE

- Bytes live in Redshift's own storage
- Redshift wrote it and owns it
- Sorted + dist-keyed + compressed
- Nothing outside Redshift is touched

`gold.unified_sales · built by dbt`

EXTERNAL TABLE — SPECTRUM

- Bytes stay in S3 — never copied in
- Redshift stores a pointer, not data
- Columns + types come from Glue
- Dropping it deletes zero data

`silver_external.sap_zsdcc`

2 · WHERE THE COMPUTE HAPPENS

THE SCAN HAPPENS OUTSIDE

Spectrum is a separate AWS-managed fleet.
It reads, filters and projects the S3 files;
only the surviving rows reach your RPUs,
which do the join and the final aggregate.

Push filters DOWN — no WHERE = read it all.

3 · THE STATEMENT — AND THE BILL

One statement mounts the whole Silver layer

`run_dbt.py:151-155`

```
CREATE EXTERNAL SCHEMA IF NOT EXISTS silver_external
FROM DATA CATALOG
  DATABASE '{silver_glue_db}'    -- silver_dev
  IAM_ROLE '{redshift_role_arn}' -- the Redshift role
```

No CATALOG_ID on purpose — Spectrum hardcodes catalogId=<account> anyway (run_dbt.py:144-146).

WHAT IT COSTS

- Spectrum is billed per terabyte scanned
- A native scan is already paid for (RPUs)
- Parquet + partition pruning = fewer bytes
- 9 models over one staging view = 9 scans

So: materialise once, then read it many times.

IN PLAIN ENGLISH

A native table is a book on your shelf. An external table is a library card — the library keeps the book and bills you per page.

L12 · Reading Data Redshift Doesn't Own

Slide: `_render/L12-redshift-spectrum.html`

The point

Redshift can query data it does not store. That capability is called **Redshift Spectrum**, and it is the only reason our Gold build can read the Silver lake without a copy step. You register an *external schema* that points at a Glue Data Catalog database; from then on, `silver_external.sap_zsdcc` behaves like a table in every SQL statement you write — but the bytes never leave S3.

Key ideas

- **Native table** = Redshift owns the bytes (its own managed, sorted, dist-keyed storage). **External table** = Redshift owns only a pointer plus a schema; the files stay in S3. Dropping an external table deletes zero data.
- The schema for an external table comes from the **Glue Data Catalog**, not from Redshift. Redshift never learned the columns — it looked them up.
- `CREATE EXTERNAL SCHEMA … FROM DATA CATALOG DATABASE '<db>' IAM_ROLE '<role>'` mounts an entire catalog database in one statement. It is idempotent with `IF NOT EXISTS`, so our runner re-issues it on every build.
- **Compute is split.** The scan/filter/project happens on an AWS-managed Spectrum fleet outside your workgroup; only surviving rows come back to your RPUs, which do the joins and the final aggregate. Push predicates down or Spectrum reads everything.
- **Cost model differs.** Native scans are paid for in RPU-seconds you're already burning. Spectrum is billed **per byte scanned in S3** — so Parquet, column pruning and partition pruning are money, not style.
- Nine models selecting from the same external-backed staging view means **nine Spectrum scans**. Materialise once, read many times.
- We deliberately do **not** pass `CATALOG_ID`: Spectrum hardcodes `catalogId=<account>` regardless (CloudTrail-confirmed 2026-05-18), which is why we mount a default-catalog database rather than the federated one.

Words you'll hear

WORD	WHAT IT MEANS HERE
Spectrum	Redshift's query layer over data stored outside Redshift
External schema	A Redshift schema whose table definitions come from Glue
External table	A table Redshift can read but does not store or own
Native table	A table in Redshift's own managed storage
Predicate pushdown	Sending the `WHERE` to the scanner so fewer bytes come back
Bytes scanned	The Spectrum billing unit — not rows, not queries

In this repo

- `src/glue/glue_engine/jobs/_scripts/run_dbt.py:151-155` — the literal `CREATE EXTERNAL SCHEMA IF NOT EXISTS silver_external FROM DATA CATALOG DATABASE … IAM_ROLE …` we issue before every dbt build, via the Redshift Data API.
- Same file, `:110-147` — the docstring explaining *why* the Data API and not `redshift_connector` (socket timeout during a 30–90 s Lake Formation / Glue resolution), and `:144-146` on omitting `CATALOG_ID`.
- `infra/modules/catalog_federation/main.tf` — the mirror databases Spectrum actually reaches.

L12 · Reading Data Redshift Doesn't Own

Do this

1. Open `run_dbt.py`, find `_ensure_silver_external_schema`, and read the SQL string it builds. Note that the schema name is hard-coded and the database name is an argument.
2. In Redshift, run `SELECT * FROM svv_external_schemas;` then `SELECT * FROM svv_external_tables WHERE schemaname = 'silver_external';` — confirm the tables exist with no data in Redshift.
3. Run `SELECT count(*) FROM silver_external.sap_zsdcc;` and then the same against `gold.unified_sales`. Compare the query plans (`EXPLAIN`) — spot the `S3 Seq Scan`.

You've got it when you can…

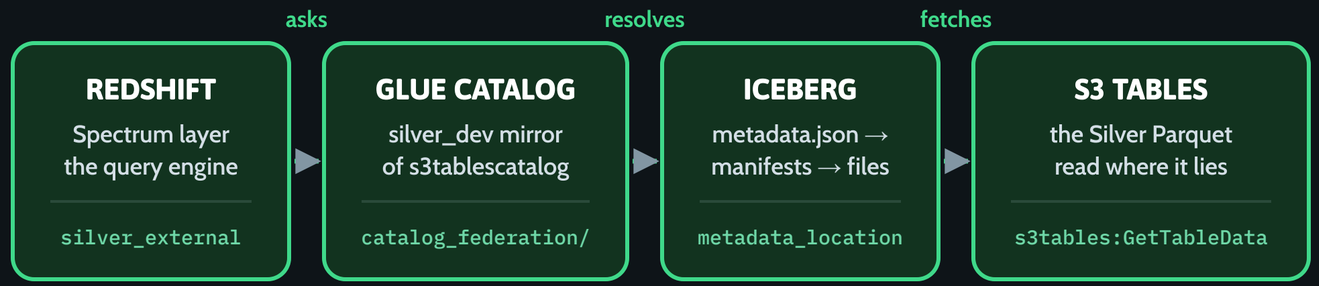
…explain, without notes, what physically happens between `SELECT … FROM silver_external.sap_zsdcc` and rows appearing — which component holds the schema, which component reads the bytes, which component does the join, and who gets the bill.

MODULE 1 · LESSON 13

Is Spectrum Still Involved?

Yes — for Silver. No — for Gold. The dividing line is the dbt build, and this is exactly where it falls.

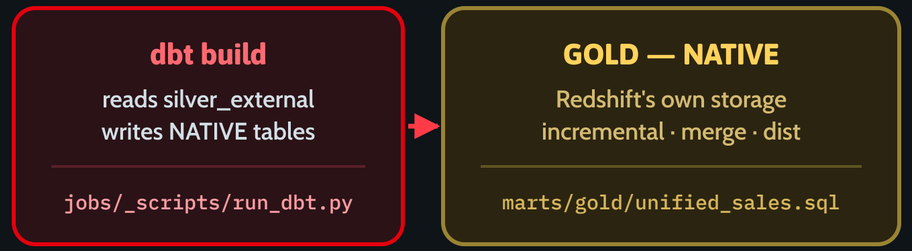
1 · YES — SILVER IS READ IN PLACE



Rows stream back into Redshift's memory — nothing is copied, nothing is stored.

SPECTRUM STOPS HERE

2 · NO — GOLD IS NATIVE



Power BI hits native storage — that's why Gold is fast.

3 · THE EVIDENCE IN THE REPO

STAGING VIEWS ARE LATE-BINDING

A plain Redshift view cannot SELECT from an external schema — Redshift refuses. So the whole staging layer is declared:

```
staging:  +materialized: view
          +bind: false   # NO SCHEMA BINDING
```

src/dbt/dbt_project.yml:63-72

TWO DIFFERENT DOORS

Spectrum never gets raw s3: actions on the S3 Tables managed bucket. Glue's Iceberg writer is the one that does.

```
READ   redshift-role → s3tables:GetTableData
WRITE  glue-role     → s3:Get/PutObject
```

infra/modules/iam/main.tf:309-317, 334

THE ONE-LINE RULE

Spectrum's job is to FEED dbt.
dbt's output is NATIVE.

After dbt — Gold, the reporting views, Power BI — no Spectrum.

That is why Gold is fast.

IN PLAIN ENGLISH

Spectrum lets Redshift read someone else's filing cabinet. dbt copies what it needs into Redshift's own cabinet — then no more trips.

L13 · Is Spectrum Still Involved? ★

Slide: `_render/L13-spectrum-iceberg-s3tables.html`

The point

This is the question everyone asks once they've seen L12, and the answer has two halves.

- **YES for Silver.** Redshift reads the Silver Iceberg tables **in place**: Redshift → Glue Data Catalog → Iceberg metadata → `s3tables:GetTableData`. Nothing is copied. Rows stream into Redshift's memory for the duration of the query and are gone afterwards.
- **NO for Gold.** dbt materialises Gold as **native Redshift tables**. Once a row is in Gold, Spectrum is out of the picture — Power BI queries Redshift's own storage. *That is why Gold is fast.*
- **The dividing line is the dbt build.** Spectrum's job is to feed dbt; dbt's output is native.

Key ideas

- The read path has four hops, and each one is a different service: the **query engine** (Redshift/Spectrum), the **catalog** (Glue — what tables exist and where their metadata lives), the **table format** (Iceberg — `metadata.json` → manifests → data files), and the **storage** (S3 Tables — the Parquet).
- Nothing in that chain is a copy. The catalog holds a `metadata_location` pointer; Iceberg holds file lists; S3 Tables holds bytes.
- Because Spectrum's Glue lookup hardcodes `catalogId=<account>`, the federated `s3tablescatalog` sub-catalogs are unreachable from Spectrum. Our Glue writer therefore **re-registers each Iceberg table into a default-catalog mirror database** (`silver_<env>`) after every write, so Spectrum can find it. Same tables, reachable name.
- **Evidence #1 — late-binding views.** A plain Redshift view cannot `SELECT` from an external schema; Redshift rejects it outright. So every staging model is declared `+bind: false` (`WITH NO SCHEMA BINDING`). That config exists *only* because staging reads `silver_external.*`. It is the fingerprint of Spectrum in the dbt project.
- **Evidence #2 — the IAM paths differ.** Spectrum's role uses `s3tables:GetTableData` and needs **no** raw `s3:` on the managed physical bucket. Glue's Iceberg S3FileIO writer is the one calling raw `s3:GetObject` / `PutObject` there. Two different doors into the same data.
- Downstream of dbt — `gold.*`, `reporting.vw_*`, Power BI — there is no Spectrum, no Glue lookup and no per-byte S3 bill.
- Practical consequence: a slow Gold build is usually a Spectrum problem; a slow *report* is never one.

Words you'll hear

WORD	WHAT IT MEANS HERE
In place	Read where it lies — no copy, no load step
Materialise	Write the query result out as a real stored table
Late-binding view	A Redshift view that resolves its source at query time, not create time
Mirror database	`silver_<env>` in the default Glue catalog, re-registered after each write
`s3tables:GetTableData`	The S3 Tables data-plane action Spectrum uses instead of `s3:GetObject`
Federated catalog	`s3tablescatalog` — where S3 Tables auto-mount, but Spectrum can't see

L13 · Is Spectrum Still Involved? ★

In this repo

- `src/dbt/dbt_project.yml:63-72` — the staging block with `+materialized: view`, `+bind: false` and the comment recording the 2026-06-05 failure ("External tables are not supported in views") that forced it.
- `infra/modules/iam/main.tf:309-317` — the comment stating Spectrum uses `s3tables:GetTableData` "so we do NOT need `s3:*` on this role", and `:334` — the explicit action list. Contrast `:64-79`, the Glue role's `s3tables:Put*` data-plane statement.
- `src/glue/glue_engine/writers/s3_tables.py:451-536` — `_register_in_mirror_catalog`: read `metadata_location`, delete, re-create the Glue table so Spectrum sees the newest snapshot.
- `src/dbt/models/marts/gold/unified_sales.sql:76-83` — `materialized='incremental'`, `dist='date'`, `sort=['date','site']`: unmistakably a native table.

Do this

1. Open `dbt_project.yml`, delete `+bind: false` in your head, and predict the exact error. Then read the comment above it and check you were right.
2. Trace one table end to end: `silver.sap.zsdcc` (S3 Tables) → mirror registration in `s3_tables.py` → `silver_external.sap_zsdcc` → `stg_sap_zsdcc` (late-binding view) → `gold.unified_sales` (native). Say out loud where Spectrum stops.
3. In Redshift, run `EXPLAIN` on a query against `stg_sap_zsdcc` and on one against `gold.unified_sales`. Only one plan mentions S3.

You've got it when you can…

…answer "is Spectrum still involved?" with **"for Silver yes, for Gold no, and the boundary is the dbt build"** — and then point at two independent pieces of evidence in the repo that prove it (`+bind: false`, and the `s3tables:GetTableData` vs raw `s3:` split in IAM).

MODULE 1 · LESSON 14

dbt: SQL as Engineered Code

A model is one SELECT. ref() builds the DAG. The warehouse then builds itself, in dependency order.

1 · A MODEL IS ONE SELECT

One SELECT. No DDL.

You write the SELECT. dbt writes the CREATE TABLE, the DROP, the atomic swap and the grants — for every model, the same way.

```
{{ config(materialized='view') }}
FROM {{ source('silver','sap_zsdcc') }}
```

models/staging/stg_sap_zsdcc.sql:17-32

2 · ref() BUILDS THE DAG

ref() is the dependency

ref('stg_sap_zsdcc') compiles to the real relation AND records an edge in the graph. dbt sorts the graph and builds in order —

```
stg_sap_zsdcc → unified_sales → vw_abv
dbt build   # deps + run + test, in order
```

9 staging · 4 dims · 5 gold · 11 reporting

3 · TESTS MAKE IT SOFTWARE

Tests ship with the model

not_null · unique · accepted_values live in YAML beside the model. dbt build runs each test right after its model and stops.

```
tests:  +store_failures: true
        +severity: error
```

src/dbt/dbt_project.yml:118-120

MATERIALIZATION IS ONE LINE OF CONFIG — src/dbt/dbt_project.yml:57-98

staging

```
+materialized: view
+bind: false
+schema: staging
```

1:1 with Silver. Renames + casts.
9 models · late-binding (see L13)

intermediate

```
+materialized: ephemeral
+schema: intermediate
```

Compiles into a CTE — no object is created at all.

marts / gold

```
+materialized: incremental
+incremental_strategy: merge
+on_schema_change: fail
```

Native Redshift tables.
4 dims · 5 fact tables.

marts / reporting

```
+materialized: view
+schema: reporting
+tags: [reporting]
```

What Power BI queries.
10 views + 1 materialised.

IN PLAIN ENGLISH

dbt is **make** for SQL — you declare what each table selects, it works out the build order, runs it, then tests the result.

L14 · dbt: SQL as Engineered Code

Slide: `_render/L14-dbt.html`

The point

dbt is `make` for SQL. You write one `SELECT` per model and nothing else — no `CREATE TABLE`, no `DROP`, no swap, no grants, no build script. dbt reads the `ref()` calls in your SQL, derives the dependency graph, builds every model in topological order, and runs each model's tests immediately after it. The result is that analyst SQL becomes reviewable, testable, version-controlled software.

Key ideas

- **A model is one file containing one `SELECT`.** The file name is the relation name. That's the whole contract.
- **`ref()` does two jobs at once**: it compiles to the real schema-qualified relation *and* it declares an edge in the DAG. You never maintain a build order by hand — you can't get it wrong, because there's nowhere to write it down.
- **`source()`** is the same idea for things dbt doesn't build — our Silver tables, declared in `models/sources.yml` against the `silver_external` schema.
- **Materialization is one line of config, not a rewrite.** The same `SELECT` can be a `view`, a `table`, `incremental`, or `ephemeral`. Changing it is a config edit; the SQL is untouched. Ours: staging = view (late-binding), intermediate = ephemeral (compiles to a CTE, creates no object), marts/gold = incremental with `merge`, marts/reporting = view.
- **`+on_schema_change: fail`** on Gold: we never silently absorb schema drift. A new or missing column stops the build instead of quietly changing a number in a report.
- **Tests are declared next to the model** (`not_null`, `unique`, `accepted_values`, plus singular SQL tests). `dbt build` interleaves run and test per node, so a failing test blocks its dependents rather than being discovered downstream. `+store_failures: true` keeps the offending rows so you can look at them.
- Everything is a text file in git: SQL, tests, materialization, docs. Reviewable in a pull request like any other code.

Words you'll hear

WORD	WHAT IT MEANS HERE
Model	One `.sql` file = one `SELECT` = one relation
`ref()` / `source()`	Dependency declaration; builds the DAG
DAG	The dependency graph dbt derives and builds in order
Materialization	How a model is persisted: view / table / incremental / ephemeral
Incremental	Build only new/changed rows, merged into the existing table
`dbt build`	deps + seed + run + test, in dependency order
Late-binding	`WITH NO SCHEMA BINDING` — required over external schemas (L13)

L14 · dbt: SQL as Engineered Code

In this repo

- `src/dbt/dbt_project.yml:57-98` — the per-layer materialization config: `staging` (view + `bind: false`), `intermediate` (ephemeral), `marts` (table), `marts.gold` (incremental / merge / `on_schema_change: fail`), `marts.reporting` (view). `:118-120` — test defaults.
- `src/dbt/models/` — 9 staging models, 4 dims, 5 Gold facts, 11 reporting views (10 views + 1 materialized view).
- `models/staging/stg_sap_zsdcc.sql:17-32` — a model in its entirety: one config line, one `SELECT`, one `source()`.
- `models/marts/gold/unified_sales.sql:76-83` — the incremental config with `unique_key`, `dist` and `sort`.
- `src/glue/glue_engine/jobs/_scripts/run_dbt.py` — how `dbt build` is actually invoked in production (Glue Python-shell, IAM-token auth, per-model run rows written to DynamoDB).

Do this

1. Open `stg_sap_zsdcc.sql`. Count the lines that are *not* the `SELECT`. There is one.
2. Follow one `ref()` chain by hand: `stg_sap_zsdcc` → `unified_sales` → `vw_sales_actuals_pyc`. Now change the order in your head and convince yourself dbt would refuse.
3. Change a staging model's materialization from `view` to `table` in `dbt_project.yml` (locally). Note that no SQL changed.
4. Read `_gold.yml` and find a test whose failure would have caught a real reporting bug.

You've got it when you can…

…take a 40-line analyst query, split it into two dbt models joined by `ref()`, pick the right materialization for each, add the one test that protects the grain, and explain why that is better than a stored procedure.

MODULE 1 · LESSON 15

One Engine, N YAML Files

We did not write 58 download jobs — we wrote one job and 58 specs. Adding a table is a YAML file, not a deployment.

1 · THE SPEC IS THE CONTRACT

The whole job, declared

specs/download/sap_mara.yaml

```yaml
table: sap.mara
source_type: sap_hana
source_config:
  jdbc_schema: SAPHANADB
  jdbc_table: MARA
  watermark_column: LAEDA        # pull only the delta
  jdbc_hash_partitions: "8"      # 8 parallel readers
landing_prefix: "sap/sap.mara"
schema:
  - { name: MATNR, type: string, pii_class: internal }
```

2 · ONE ENGINE, MANY SPECS

ONE ENGINE

src/glue/glue_engine/

Four jobs cover every table:

source_download · bronze_pull
bronze_to_silver · abap_transform

145 spec files

58 dl · 67 bronze · 6 slv · 14 tf

3 · THE SPEC IS VALIDATED

PYDANTIC IS THE GATE

glue_engine/spec.py

A spec is parsed into a typed model — extra='forbid', so a typo is an error, not silence.

Refused at load time:

- merge_key not in the schema
- a PII column used as a key

ADDING THE 59th TABLE — CONFIG vs CODE

THE WAY WE DIDN'T

one job per table

- One Python job per table — 145 near-identical scripts to drift apart
- A Terraform resource and a deploy pipeline run for every new table
- Reviewers read code to find out which column is the watermark

THE WAY WE DO

specs/bronze/sap_zncr01.yaml:18

- Add specs/download/sap_xyz.yaml — the engine already knows how
- The Bronze spec adds one line: merge_key: [MANDT, DATUM, WERKS]
- That single line stopped 7,966 duplicate rows in bronze.sap.zncr01

IN PLAIN ENGLISH

The engine is the printer; the YAML is the document. You don't build a new printer every time you want to print a different page.

L15 · Spec-Driven Design — One Engine, N YAML Files ★

Slide: `_render/L15-spec-driven-design.html`

The point

We ingest 58 source tables and we did **not** write 58 jobs. We wrote **one engine** and **145 spec files**. A spec is a YAML document that declares *what* a table is — its source, its key, its watermark, its columns — and the engine works out *how*. Adding table #59 is a pull request against a directory of data, not a new script, not a new Terraform resource, not a deployment.

Key ideas

- **The spec is a contract, not a config file.** It is the single source of truth for a table's schema and load behaviour, it lives in git, and it is reviewed as data — 30 declarative lines instead of 200 lines of near-duplicate Python.
- **Config vs code.** Anything that varies per table (schema, keys, parallelism, watermark) is config. Anything that is the same for every table (JDBC read, typing, dedup, Iceberg MERGE, audit columns, watermark advance) is code, written once.
- **One job per table is the failure mode we avoided.** 145 near-identical scripts drift: one gets a bug fix, the other doesn't. With one engine, a fix lands everywhere at once — and a regression is equally global, which is why the engine is the thing that carries the tests.
- **The spec is validated, not just parsed.** Pydantic models with `extra="forbid"` mean a typo'd key is an *error*, not a silently ignored line. Cross-field validators refuse specs that would corrupt data: a `merge_key` naming a column that isn't in the schema, a `pii_class: pii` column used as a merge key (masking runs before the write, so the key would stop matching SAP), a `landing_prefix` on the wrong kind of spec.
- **The fields that matter.** `schema` (the column contract, types + `pii_class`), `watermark_column` (pull only the delta), `merge_key` (the natural key for the Iceberg upsert), `jdbc_hash_partitions` (parallel readers for the JDBC pull).
- **One line can be worth thousands of rows.** `bronze.sap.zncr01` held 446,611 rows against a 438,645-row source after overlapping appends. Adding `merge_key: [MANDT, DATUM, WERKS]` to the Bronze spec turned the append into an upsert and stopped the duplication — a data-correctness fix delivered as config.
- **This is what makes the architecture portable.** Next engagement, 8 Oracle sources: new connector class, same engine, new YAML. The spec shape is the reusable asset.

Words you'll hear

WORD	WHAT IT MEANS HERE
Spec	One YAML file describing one table
Spec-driven / meta-design	Designing the *description format*, then one engine that reads it
Watermark	The column that says "what changed since last time"
Merge key	The natural key the Iceberg `MERGE` upserts on
Hash partitions	How many parallel JDBC readers pull the table
`extra="forbid"`	Pydantic setting that turns an unknown key into an error
P1 / P2 spec	Download spec (source → raw S3) vs load spec (raw → Bronze)

L15 · Spec-Driven Design — One Engine, N YAML Files ★

In this repo

- `src/glue/specs/download/sap_mara.yaml` — a complete P1 spec: `table`, `source_type: sap_hana`, `jdbc_schema` / `jdbc_table`, `watermark_column: LAEDA`, `jdbc_hash_field: MATNR`, `jdbc_hash_partitions: "8"`, `landing_prefix`, and the 16-column `schema` block with `pii_class` per column.
- `src/glue/specs/bronze/sap_zncr01.yaml:18` — `merge_key: [MANDT, DATUM, WERKS]`, with the comment recording the 446,611-vs-438,645 duplication it fixed.
- `src/glue/glue_engine/spec.py` — `BronzeSpec`, `SilverSpec`, `TransformSpec`. Validators at `:188-201` (merge_key ⊆ schema), `:203-223` (no PII merge key), `:225-259` (P1-vs-P2 spec shape).
- `src/glue/glue_engine/` — the four jobs that consume all of it: `source_download`, `bronze_pull`, `bronze_to_silver`, `abap_transform`.
- Spec counts today: **58** download · **67** bronze · **6** silver · **14** transform.

Do this

1. Read `sap_mara.yaml` top to bottom, then `sap_zncr01.yaml`. List every field that differs and decide, for each, whether it is config or code.
2. Open `spec.py` and find `_validate_pii_not_merge_key`. Read the docstring — it explains a data-corruption bug that can no longer be written.
3. Write (don't deploy) a download spec for a new SAP table: pick the watermark column, the hash field, and the merge key. Have a neighbour try to break it.
4. Try adding an unknown key like `parallelism: 8` and predict what Pydantic does.

You've got it when you can…

…be handed one Oracle table from the next client and produce its download + bronze spec YAML from scratch, correctly choosing the watermark column, the merge key and the partition count — and explain why that is the whole job, with no new code to write and nothing to deploy.

MODULE 1 · LESSON 16

SAP → RAW: Land It, Don't Touch It

P1 source_download pulls from HANA over JDBC and writes Parquet. Nothing is cleaned, typed or renamed.

1 · WHY RAW EXISTS AT ALL

REPLAY, AUDIT, PARITY

the layer you never edit

- Re-run any day without touching SAP
- Bronze bug? Fix code, replay the files
- Proof of exactly what the source sent
- Versioned, Glacier-tiered, 7-year hold
- Never edited. Never deleted.

`jobs/source_download.py`
`infra/env/dev/source_download.tf`

2 · HOW WE READ SAP

JDBC, IN PARALLEL

one call to SAP, per table per day

- Custom driver ngdbc.jar, not a Glue one
- Full load · watermark delta · date range
- jdbc_hash_partitions: 8 readers at once
- 638M-row S611 refuses a 1-partition read
- Read once. Never read again for this day.

`sources/sap_hana.py`
`specs/download/sap_zncr01.yaml`

3 · WHAT LANDS ON S3

ONE PREFIX PER CYCLE

parquet parts, then the marker

- Data files first, _SUCCESS marker last
- No marker means P2 refuses the cycle
- Re-run overwrites that cycle in place
- Big windows land under chunk=from_to
- Two overlapping writers → hard failure

`writers/raw_landing.py`
`raw/sap/sap.zncr01/cycle=2026-08-10/`

WHAT CHANGES HERE

NOTHING. THAT IS THE JOB.

Byte-for-byte what SAP returned. Not typed, not renamed,
not filtered — only landed, and partitioned by cycle.
The one addition: four audit columns stamped on the way past.

THE MANDT TRAP

Every SAP table is split by client.

MANDT is the first key field of almost every SAP table. Client
100 is productive; 200 is a test client sitting in the same table.
Forget WHERE MANDT = '100' and test rows land inside your facts.

`sources/sap_hana.py:305` · `specs/download/sap_zncr01.yaml:27`

IN PLAIN ENGLISH

Take the photocopy before you edit. If the edit goes wrong the original is still there — and SAP is never asked for it twice.

L16 · SAP → RAW: Land It, Don't Touch It

Slide: `_render/L16-sap-to-raw.html`

The point

The first hop is the one where you do **nothing**. P1 (`source_download`) opens one JDBC connection to SAP HANA, reads the rows it is allowed to read, and writes them to S3 as Parquet — byte-for-byte, no types, no renames, no business rules. The only thing that changes is *where the bytes live* and *which cycle folder they live in*. Everything downstream can then be rebuilt from those files without ever asking SAP again.

Key ideas

- **Raw exists so you can replay.** A bug in Bronze, Silver or Gold costs a re-run over files you already have — not another extract window, not another conversation with the SAP team.
- **Raw exists so you can prove.** When a number is disputed, raw is the evidence of exactly what the source returned that day. It is versioned, Glacier-tiered and held for seven years, and it is never edited or deleted.
- **Three read modes, one connector.** Full load (`read_full`), watermark delta (`read_incremental`), and a bounded date window (`read_range`) for backfills and chunked initial loads. The mode is chosen per run, from config — not by editing code.
- **JDBC reads are partitioned on purpose.** `jdbc_hash_partitions: 8` splits one logical read into eight parallel connections. A table declaring `expected_row_count` above `single_partition_max_rows` and *no* partitioned read is **refused at configure time** — a one-executor read of 638 M rows OOMs or times out.
- **`_SUCCESS` is the commit.** Data files are written first, the marker last. If the job dies mid-write there is no marker, and the P2 reader refuses the prefix. A half-written cycle can never look "done".
- **The MANDT filter is not optional.** SAP tables are partitioned by client; without `WHERE MANDT = '100'` you silently ingest the test client's rows into your facts. The connector raises unless a spec supplies `client` or declares `client_independent: true` .
- **The one thing raw does add:** four audit columns (`_source_system` , `_source_table` , `_ingested_at` , `_run_id`) plus `_watermark_valid` , stamped on the way past so every row can be traced back to the run that fetched it.

Words you'll hear

WORD	WHAT IT MEANS HERE
P1	The download half of the pipeline — the only code that talks to SAP
Cycle	One dated run of the pipeline; raw is partitioned as `cycle=<id>`
`_SUCCESS`	The marker object that says "this cycle is complete and readable"
Watermark	The high-water value (e.g. a date) marking where the last read stopped
MANDT	SAP's client field — which company/environment a row belongs to
Partitioned read	Splitting one JDBC read into N parallel range/hash queries
Landing prefix	The relative S3 path a table lands under, qualified at runtime

L16 · SAP → RAW: Land It, Don't Touch It

In this repo

- `src/glue/glue_engine/jobs/source_download.py` — the whole P1 job: resolve spec → pick read mode → call the connector → `write_raw` → advance the watermark.
- `src/glue/glue_engine/sources/sap_hana.py:305-361` — the MANDT productive-client filter (G02) and the partitioned-read configuration (G01), with the safety threshold right below at `:363-411`.
- `src/glue/glue_engine/writers/raw_landing.py` — `resolve_landing_base`, `write_raw`, the marker-last ordering, and `_assert_single_write` (two overlapping Spark writes under one prefix = hard failure, with the live 749 M-vs-679 M row incident in the docstring).
- `src/glue/specs/download/sap_zncr01.yaml` — a complete download spec: `client: "100"`, `watermark_column: DATUM`, `safety_buffer_days: 7`, `jdbc_hash_partitions: "8"`, `landing_prefix`.

Do this

1. Open `sap_zncr01.yaml` and `sap_s611.yaml` side by side. Find every field that differs, and say out loud what each one changes about the read.
2. In `source_download.py`, trace the three branches that pick `read_range` / `read_full` / `read_incremental`. Which job arguments decide it?
3. In `raw_landing.py`, delete (mentally) the `_assert_single_write` call and describe the failure it would let through.
4. List a landed cycle prefix in S3 and find the `_SUCCESS` object. Open it — it is a JSON manifest, not an empty file.

You've got it when you can…

…explain why a bug found in Gold on a Tuesday does **not** require a new extract from SAP, and point at the exact two lines of `write_raw` that guarantee a crashed job never leaves a cycle that looks complete.

MODULE 1 · LESSON 17

RAW → BRONZE: Typed, Deduped, Idempotent

P2 bronze_pull never calls SAP. It reads the landed cycle back and MERGEs it into an Iceberg table.

1 · WHY P1 AND P2 ARE SPLIT

P1 PULLS · P2 LOADS

two jobs, one blast radius

- P1 owns the only call to SAP
- P2 reads S3 — never the source
- Load bug? Replay from raw, for free
- Retries can never hammer HANA
- P1 owns the watermark; P2 never moves it

`jobs/bronze_pull.py`
`sources/landed_files.py (s3_landing)`

2 · WHAT P2 ACTUALLY DOES

TYPE, DEDUPE, MERGE

the spec is the contract

- schema: in the YAML applies real types
- PII columns hashed before any write
- dropDuplicates on merge_key first
- Audit columns carried through, not redone
- Zero business logic. None. Ever.

`specs/bronze/sap_zncr01.yaml`
`_source_system _source_table _ingested_at`

3 · WHY REPLAYS ARE SAFE

MERGE, NOT APPEND

Iceberg upsert on the natural key

- WHEN MATCHED → UPDATE SET *
- WHEN NOT MATCHED → INSERT *
- Same input twice = same row count
- Watermark re-reads a 7-day safety window
- The overlap is absorbed, not duplicated

`writers/s3_tables.py (merge_into)`
`merge_key: [MANDT, DATUM, WERKS]`

WHAT CHANGES HERE

SHAPE, NOT MEANING.

Text becomes numbers and dates. Duplicate keys collapse to one row. Audit columns ride along. No renames, no joins, no rules — Bronze is still one row per source row.

THE WAR STORY · 2026-07-14

446,611 rows in Bronze. 438,645 at source.

bronze.sap.zncr01 was written with plain append, so overlapping loads re-inserted the same keys. Nothing errored. Nothing warned. Declaring merge_key turned the reload into an upsert — and it stopped.

`specs/bronze/sap_zncr01.yaml:18 · writers/s3_tables.py:193`

IN PLAIN ENGLISH

Append is INSERT; merge is INSERT … ON CONFLICT UPDATE. Run it twice, the table looks identical — that is idempotency.

L17 · RAW → BRONZE: Typed, Deduplicated, Idempotent

Slide: `_render/L17-raw-to-bronze.html`

The point

P2 (`bronze_pull`) reads the cycle P1 landed and writes it into an Iceberg table. It applies real types, collapses duplicate keys, and **upserts** instead of appending — so running it twice produces the same table, not twice the rows. What it never does is business logic. Bronze is still one row per source row; only its *shape* has changed.

Key ideas

- **P1 and P2 are separate jobs on purpose.** P2 reads S3 via the `s3_landing` connector and never touches the source. Retries, replays and Glue's auto-retry therefore cannot hammer HANA, and a load bug is free to fix.
- **One owner per fact.** P1 owns the watermark; `bronze_pull` explicitly forces `new_watermark = watermark_value` on the landed-read path so the reader's contract is guaranteed by the job, not merely promised by the connector.
- **Types come from the spec, not from inference.** The `schema:` block in `specs/bronze/*.yaml` is the contract; the same file names the PII columns, which are hashed *before* any write.
- **`merge_key` makes replays free.** Iceberg `MERGE INTO … WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *` keyed on the natural key. Same input twice → same row count.
- **Deduplicate before you merge.** Raw can legitimately hold duplicate keys (a retried P1 write), and Iceberg refuses to match one target row to two source rows (`MERGE_CARDINALITY_VIOLATION`). So `dropDuplicates(merge_key)` runs first.
- **Overlap is a feature.** `safety_buffer_days: 7` deliberately re-reads a trailing window, because a business date is not a change timestamp — a restated store-day keeps its original date. The MERGE absorbs the overlap.
- **The war story.** `bronze.sap.zncr01` held **446,611** rows against a **438,645**-row source (2026-07-14). Plain `append` plus overlapping loads re-inserted the same keys; nothing errored and nothing warned. Declaring `merge_key: [MANDT, DATUM, WERKS]` stopped new duplicates — the existing ones needed a separate one-time dedup, which is the expensive part.

Words you'll hear

WORD	WHAT IT MEANS HERE
P2	The load half — reads landed files, writes Bronze
Idempotent	Running it again changes nothing you can observe
MERGE / upsert	Update the row if the key exists, insert it if it doesn't
`merge_key`	The natural (source) primary key the upsert matches on
Natural key	The key the business already has — not a surrogate id
CDC	Change data capture: only rows that changed since last time
Audit column	`_run_id` , `_ingested_at` etc. — provenance, not data

L17 · RAW → BRONZE: Typed, Deduplicated, Idempotent

In this repo

- `src/glue/glue_engine/jobs/bronze_pull.py:298-316` — the write-strategy fork: `full_refresh` for Excel, `dropDuplicates` + `merge_into` when `merge_key` is declared, `append` otherwise.
- Same file, `:114-160` — the `remote_pull` branch that swaps in the `s3_landing` reader, and `:189-194`, where P2 refuses to move the watermark.
- `src/glue/glue_engine/writers/s3_tables.py:193-281` — `merge_into`: the generated `MERGE INTO`, the delete-aware CDC arm (`_change_op = 'D'`), and the first-write `CREATE TABLE` fallback.
- `src/glue/specs/bronze/sap_zncr01.yaml:14-18` — the merge key, with the 446,611-vs-438,645 incident recorded in the comment above it.

Do this

1. Read the generated SQL in `merge_into`. Write out, by hand, the statement it produces for `sap.zncr01`.
2. Change one row's `ZCUSTOMER` in a copy of the landed Parquet, re-run the merge mentally, and state which arm fires.
3. Find `_dedupe_latest` in `s3_tables.py` and explain why the *first-write* path needs it just as much as the merge path does.
4. Open two bronze specs — one with `merge_key`, one without — and justify each choice from the source's shape.

You've got it when you can…

…explain to a colleague why re-running yesterday's load is safe, name the one YAML field that makes it safe, and describe exactly how `bronze.sap.zncr01` ended up with 7,966 rows that no SAP row justified.

MODULE 1 · LESSON 18

BRONZE → SILVER: Cleansed, Conformed, Rebuilt

Where business meaning arrives — and where one careless join silently doubles your revenue.

1 · CONFORMING

ONE NAME, ONE KEY

SAP words become business words

- KTGRM → aagm · WERKS → site
- FKDAT → date · ZAMT_SOLD → sale
- Leading zeros stripped: norm_aagm()
- Same key means the same thing everywhere
- Only now are cross-table joins legal

`glue_engine/abap/helpers.py`
`specs/transform/*.yaml`

2 · REBUILD, DON'T PULL

ABAP → PYSPARK

derived objects are re-implemented

- A QuickView is code, not a table
- We read its base tables and rebuild it
- zsdcc = ZDSALES ⋈ TVKMT (flat_join)
- basket = a ZHOCIDC receipt state machine
- scan_611 · distress_603 · id8_product

`glue_engine/abap/ops.py`
`glue_engine/abap/baskets.py`

3 · HOW SILVER IS WRITTEN

IDEMPOTENT, AGAIN

no watermark — it reads all of Bronze

- Three no-op re-runs once tripled Silver
- merge on the deduplicate key, or …
- overwrite_partitions for fact streams
- Silver keeps EVERY row — never lossy
- Scope is a Gold decision, not a Silver one

`jobs/bronze_to_silver.py`
`writers/s3_tables.py`

WHAT CHANGES HERE

BUSINESS MEANING.

Names, units and keys are conformed. SAP's own derived objects are rebuilt in PySpark. A row now means something a human can name — not just what a table happened to hold.

THE FAN-OUT TRAP · TVKMT

One missing filter doubles every sales row.

TVKMT is language-keyed (MANDT, SPRAS, KTGRM). In client 100 it holds 43 English depts + 43 Arabic + 3 German. Join on KTGRM alone and every ZDSALES row matches twice — sale and count silently 2×.

`specs/transform/zsdcc.yaml:59 → right_where: "SPRAS = 'E'"`

IN PLAIN ENGLISH

A join is a lookup. If the lookup holds two rows for one key, every left row comes back twice — and nothing raises an error.

L18 · BRONZE → SILVER: Cleansed, Conformed, Rebuilt

Slide: `_render/L18-bronze-to-silver.html`

The point

This is the hop where **business meaning is added**. Column names stop being SAP field codes and start being words (`KTGRM` → `aagm` , `FKDAT` → `date` , `ZAMT_SOLD` → `sale`); keys are normalised so the same value means the same thing in every table; and SAP's *derived* objects — QuickViews, extractors, report logic written in ABAP — are **rebuilt in PySpark** rather than pulled, because they are code, not tables. Silver's job is fidelity: it keeps every row it was given.

Key ideas

- **Conforming is a contract, not cosmetics.** Until `aagm` means the same thing in `zsdcc` , `dim_dept` and the budget upload, no join across them is trustworthy. `norm_aagm()` strips leading zeros so `'09'` and `'9'` stop being two departments.
- **Derived objects are rebuilt, not read.** A SAP QuickView has no table behind it. We read its *base* tables from Bronze and re-implement the join/fold in Spark — `zsdcc = ZDSALES ⋈ TVKMT` , `basket` = a receipt state machine folded over `ZHOCIDC` , plus `scan_611` , `distress_603` , `id8_product` , `dim_dept` .
- **The correctness-heavy logic lives in pure functions.** `abap/baskets.py` and `abap/helpers.py` are unit-tested plain Python; `abap/ops.py` is the thin Spark adapter that calls them. You can test the business rule without a cluster.
- **Silver has no watermark.** `bronze_to_silver` reads *all* of Bronze every run, so plain `append` amplifies: three no-op re-runs once tripled Silver. Two idempotent modes replace it — `merge` on the `deduplicate` op's key, or `overwrite_partitions` for the fact streams that have no row-level key.
- **Silver is permissive; Gold is where scope lives.** When an inner join to TVKMT silently discarded 273,867 ZDSALES rows (SAR 2.29 bn, 12.43 % of value), the fix was a LEFT join — because once Silver drops a row, Gold never sees Bronze again and the row is gone for good.
- **The trap: language fan-out.** TVKMT is keyed `(MANDT, SPRAS, KTGRM)` . In client 100 it holds 43 English departments, 43 Arabic and 3 German. Join on `KTGRM` alone and **every** sales row matches ~2×, silently doubling sale and customer count. `right_where: "SPRAS = 'E'"` restores 1:1 — and it mirrors what the QuickView's own selection screen did.

Words you'll hear

WORD	WHAT IT MEANS HERE
Conform	Make names, units and keys mean the same thing everywhere
Fan-out	A join that returns more rows than it started with
ABAP	SAP's programming language — where the derived logic lives
QuickView	A SAP-side saved query; code, not a stored table
Grain	What one row represents (site × date × dept, …)
`overwrite_partitions`	Replace exactly the partitions in this batch, keep the rest
Idempotent	Re-running changes nothing observable

L18 · BRONZE → SILVER: Cleansed, Conformed, Rebuilt

In this repo

- `src/glue/glue_engine/jobs/bronze_to_silver.py:192-233` — the write-mode fork and *why* `append` was removed, with the "3 re-runs tripled Silver" note.
- `src/glue/glue_engine/abap/ops.py:104-137` — `flat_join_op`, including `right_where` / `right_select` and the docstring explaining the fan-out they prevent. `basket_op` is at `:44`, `scan_611` at `:729`, `distress_603` at `:1001`.
- `src/glue/specs/transform/zsdcc.yaml:22-63` — the 🔴 FAN-OUT banner (measured live, with the 43/43/3 split) and the 🟡 LEFT-JOIN decision with the 273,867-row measurement.
- `src/glue/glue_engine/abap/helpers.py` — `norm_aagm`, `resolve_join_type`, `select_options`: the shared conforming primitives.

Do this

1. Read `zsdcc.yaml` top to bottom, including the comments. It is the single best-documented spec in the repo — the comments *are* the lesson.
2. Delete `right_where` in your head and compute the resulting row count from the 43/43/3 split. Which measures inflate, and by how much?
3. Find one op in `ops.py` that returns **two** DataFrames and explain why one input can produce two Silver tables.
4. Pick any Silver spec and identify its `write_mode`. Justify it from the table's grain.

You've got it when you can…

…explain why we rebuild a SAP QuickView instead of querying it, and describe — with numbers — how a join to a harmless little text table can silently double a revenue figure without raising a single error.

MODULE 1 · LESSON 19

SILVER → GOLD: Shaped for the Question

dbt builds star-schema tables inside Redshift, so Power BI stops computing and starts reading.

1 · THE STAR AND ITS GRAIN

FACTS + SHARED DIMS

one row per question, not per event

- Grain: site × date × dept × scenario
- scenario: '2026 A' · '2025 A' · 'Budget'
- Six UNION ALL legs, one column shape
- 5 gold facts share 4 conformed dims
- CY vs PY vs Budget = a filter, not a join

`src/dbt/models/marts/gold/`
`marts/dims/ dim_site dim_date dim_dept`

2 · HOW IT IS BUILT

dbt, INCREMENTAL MERGE

native Redshift tables, not the lake

- materialized = 'incremental'
- incremental_strategy = 'merge'
- unique_key = site, date, aagm, scenario
- Re-run a day and its rows are replaced
- dist = 'date', sort = date then site

`models/marts/gold/unified_sales.sql:76`
`gold_incremental_predicate('date')`

3 · WHAT POWER BI READS

DAX → SQL

measures computed once, then stored

- Variance, achievement %, ABV live in SQL
- 10 reporting views + 1 materialised
- One definition, not one per report
- Power BI reads native Redshift storage
- No Spectrum left on the last mile

`models/marts/reporting/vw_*.sql`
`vw_sales_actuals_pyc.sql`

WHAT CHANGES HERE

ANSWERS, NOT RECORDS.

Rows are reshaped to the question the report asks. Every measure is computed once, in SQL, and stored. Power BI reads a number instead of deriving one.

THE 'All Dept' TRAP · CRIT-04

A helpful subtotal row that doubles every KPI.

unified_sales carries a synthetic dept = 'All Dept' row per site × date, so DAX can filter straight to it. Any rollup that SUMs across dept must exclude it — or every store total comes out about 2× high.

`reporting/vw_store_performance_bands.sql:53 WHERE dept != 'All Dept'`

IN PLAIN ENGLISH

Gold is the answer sheet, not the ledger. And one subtotal row hiding inside a detail table doubles every total you compute.

L19 · SILVER → GOLD: Shaped for the Question

Slide: `_render/L19-silver-to-gold.html`

The point

Gold stops storing *records* and starts storing *answers*. dbt reshapes Silver into a star schema — facts surrounded by conformed dimensions — at exactly the grain the report asks for: one row per **site × date × dept × scenario**. Measures that used to be computed in DAX, at click time, on every user's machine, are computed once in SQL and stored. And unlike Silver, Gold is a **native Redshift table**: after the dbt build, Spectrum is out of the picture.

Key ideas

- **The grain is the design.** `gold.unified_sales` has one row per site × date × dept × scenario. Everything else — which scenarios exist, how comparisons work, what a report can slice by — follows from that sentence.
- `scenario` **turns a hard join into a cheap filter.** Current-year actuals, prior-year actuals and Budget are six `UNION ALL` legs sharing one column shape, tagged `'2026 A'` / `'2025 A'` / `'Budget'`. Comparing this year with last year is `WHERE scenario = …`, not a self-join.
- **Incremental merge, not rebuild.** `materialized='incremental'`, `incremental_strategy='merge'`, `unique_key=['site','date','aagm','scenario']`. Re-run a day and its rows are replaced, not added.
- **Sentinels beat NULLs in a merge key.** All-Dept rows carry `aagm = '__ALL__'` because `NULL = NULL` is never true in a MERGE predicate — with NULL they would re-insert on every incremental run and silently inflate the table.
- **Measures move to SQL.** Variance vs budget, achievement %, ABV are computed in the reporting views, so a measure has one definition instead of one per report, and Power BI reads a number instead of deriving one.
- **The trap:** `'All Dept'`. `unified_sales` deliberately carries a synthetic rolled-up row (`dept = 'All Dept'`) per site × date, because the existing DAX filters straight to it. Any rollup that SUMs *across* dept must exclude it — otherwise the detail rows and their own subtotal are added together and every store total comes out roughly **2× high**. That is a real defect found in this codebase (CRIT-04).
- **Know where each number comes from.** The All-Dept `sale` is `SUM(ZSDCC.sale)`; the All-Dept `cc` comes from **ZSCC directly**, because summing per-dept customer counts would double-count a basket that spans two departments.

Words you'll hear

WORD	WHAT IT MEANS HERE
Star schema	One fact table surrounded by dimension tables
Conformed dimension	One dimension shared by several facts, with one meaning
Grain	What exactly one row of a fact table represents
Scenario	Which version of reality a row describes: CY / PY / Budget
Incremental	Rebuild only the affected rows, not the whole table
`unique_key`	The columns dbt's MERGE matches an existing row on
Sentinel	A real value standing in for "all" or "none" (`'__ALL__'`)
DAX	Power BI's formula language — what we are moving out of

L19 · SILVER → GOLD: Shaped for the Question

In this repo

- `src/dbt/models/marts/gold/unified_sales.sql` — the canonical Gold fact. `:76-83` is the incremental/merge config; `:85-117` the CY/PY actual legs; `:133-211` the synthetic All-Dept CTEs (with the `'__ALL__'` sentinel rationale at `:163-170`); `:226-268` budget.
- `src/dbt/models/marts/reporting/vw_store_performance_bands.sql:48-53` — `WHERE s.dept != 'All Dept'`, the CRIT-04 fix, with the "or every store total is ~2x" comment.
- `src/dbt/models/marts/dims/` — the four conformed dimensions: `dim_site`, `dim_date`, `dim_dept`, `dim_area_mgr`.
- `src/dbt/macros/scenario_helpers.sql` — `scenario_cy()` / `scenario_py()` / `scenario_budget()`, so the year labels live in one place.

Do this

1. Read the six CTEs of `unified_sales.sql` and name, for each, which Silver/staging model it draws from and which scenario it emits.
2. Write the query that returns total sale per site for one date — first wrongly (no `dept` filter), then correctly. Predict the ratio between them before you run it.
3. Find one reporting view that *keeps* the All-Dept rows and explain why keeping them is correct there.
4. In `unified_sales.sql`, change `unique_key` to drop `scenario` and describe what breaks on the next incremental run.

You've got it when you can…

…state the grain of `gold.unified_sales` in one sentence, explain why a helpful subtotal row living inside a detail table is dangerous, and name the exact predicate that keeps every store total honest.

MODULE 1 · LESSON 20

The Last Mile: Gold → Power BI

Power BI never reads Gold. It reads nine views — and 58 of 73 DAX measures now live in SQL.

1 · THE VIEW LAYER

9 VIEWS + 1 MATERIALISED TABLE

Power BI's only contract with the lakehouse

- vw_sales_actuals_pyc
- vw_customer_count
- vw_abv
- vw_store_performance_bands
- vw_dept_performance
- vw_gross_profit
- vw_gross_profit_mtd
- vw_distress
- vw_am_kam
- all in schema reporting

mv_top5_bottom5_sites_by_dept · built as a table

src/dbt/models/marts/reporting/

2 · HOW POWER BI READS IT

STORAGE MODE

DIRECTQUERY

No copy of the data in the PBIX
Every visual emits live SQL
Freshness = last dbt Gold build

IMPORT

Would cache a copy · goes stale
Not used for the fact views

docs/design-reference/decisions/
0021-thin-powerbi-over-gold-views.md

3 · WHERE THE MEASURES LIVE

73 DAX MEASURES

58 → SQL

Pushed into the reporting views
≈ 80% of the whole measure set

15 → DAX

9 ranks: RANKX needs PBI context
6 display formatters (K / M)

Why: SQL is testable in CI. DAX isn't.
marts/reporting/*.sql

THE 07:00 KSA DAILY SLA — ALL TIMES ASIA/RIYADH (UTC+3)

06:30	07:00	07:30	08:00
Bronze landed	Silver conformed	Gold built by dbt	Alarm if not done

No dataset refresh to wait for.
DirectQuery means the number you see at 09:00 is the 06:30 pull.

IN PLAIN ENGLISH

Gold is the warehouse; the views are the shop window. Power BI just points at the window — it keeps no stock of its own.

L20 · The Last Mile: Gold → Power BI

Slide: `_render/L20-gold-to-powerbi.html`

The point

Gold is not the finish line. Power BI never points at a Gold fact table — it points at a thin **reporting view layer** that pre-computes the joins, the prior-year shift and the aggregations that used to be DAX. Getting that boundary right is what makes the reports fast, testable and reusable by anything else that shows up later.

Key ideas

- **Nine views + one materialised table** are the entire contract between the lakehouse and Power BI. Nothing else is exposed.
- **DirectQuery, not Import.** The `.pbix` caches no data; every visual emits live SQL over the Redshift ODBC driver. There is no dataset refresh to schedule or to fail.
- **58 of 73 DAX measures (~80%) were pushed down into SQL.** The rule: *if it doesn't depend on what the user clicks, it belongs in a view.*
- **15 measures stay in DAX** — 9 rank-family (`RANKX` / `ALLSELECTED` need Power BI's runtime filter context) and 6 display formatters (the K/M wrappers). That's deliberate, not unfinished.
- **Why bother:** SQL views are regression-tested in CI, change-managed in git, and readable by any future consumer. DAX in a `.pbix` is a black box.
- **Freshness is bounded by the last dbt Gold build.** Bronze 06:30 → Silver 07:00 → Gold 07:30 KSA, alarm at 08:00. Because it's DirectQuery, a page opened at 09:00 shows the 06:30 pull.
- Each view's header comment **names the DAX measures it replaced** — that's the migration audit trail.

Words you'll hear

WORD	WHAT IT MEANS HERE
Reporting view	A dbt-built SQL view in the Redshift `reporting` schema; the only thing Power BI reads
DirectQuery	Power BI storage mode where every visual queries the database live and caches nothing
Import	The opposite mode — data copied into the `.pbix`, refreshed on a schedule. Not used for facts
DAX	Power BI's measure language. `SUM(...)`, `RANKX(...)`, `DIVIDE(...)`
Push-down	Moving a calculation out of DAX and into the SQL view
Materialised	Computed once at build time and stored, instead of recomputed per query (`mv_top5_bottom5_sites_by_dept`)
Grain	The one row means *this*. e.g. Site × EquivalentDate × Dept
SLA	The promise: numbers correct and available by 07:00 KSA, daily

L20 · The Last Mile: Gold → Power BI

In this repo

- `src/dbt/models/marts/reporting/` — the whole serving layer
- `vw_sales_actuals_pyc.sql` — the headline view; 12 DAX measures, CY/PY/Budget in one row
- `vw_customer_count.sql`, `vw_abv.sql`, `vw_dept_performance.sql`, `vw_store_performance_bands.sql`
- `vw_gross_profit.sql`, `vw_gross_profit_mtd.sql`, `vw_distress.sql`, `vw_am_kam.sql`
- `mv_top5_bottom5_sites_by_dept.sql` — the hybrid; materialised as a **table**, not a Redshift MV
- `_reporting.yml` — the dbt tests and column contracts for all of the above
- `docs/design-reference/decisions/0010-power-bi-direct-query-vs-import.md` — storage-mode ADR
- `docs/design-reference/decisions/0021-thin-powerbi-over-gold-views.md` — "thin Power BI" ADR
- `docs/CLIENT-TECHNICAL-DESIGN.md` §Phase 5 — view → PBI page mapping and measure counts

Do this

1. Open `vw_sales_actuals_pyc.sql` and read only the header comment. List the 12 DAX measures it replaced.
2. Find the `COALESCE(...)` on `sale_var_vs_budget` and read the comment. Explain in one sentence why a bare `SUM(...) - SUM(...)` flipped a page total from +292 M to −8 M.
3. Find the `WHERE us.site IN (SELECT site FROM dim_site WHERE is_reporting_scope)` semi-join. Say why it is a semi-join and not a `JOIN`.
4. Open `mv_top5_bottom5_sites_by_dept.sql` and say why *this one* could not be a plain view.

You've got it when you can…

- Draw the last mile: Gold table → `reporting.vw_*` → DirectQuery → visual, and say what happens at each arrow.
- Give the rule for deciding whether a new calculation goes in DAX or in a view — and apply it to "show me last 7 days from the slicer" (DAX) vs "sale vs budget %" (view).
- Explain why there is no Power BI refresh schedule in this platform, and what *does* bound freshness instead.
- Name the three things a view buys you that a DAX measure doesn't: testable, governable, reusable.

MODULE 1 · LESSON 21

What Actually Runs It

One EventBridge rule fires. After that nothing is scheduled — every next phase is unlocked by a barrier.

1 · THE CHAIN — ONE CRON, THEN NOTHING IS SCHEDULED

EVENTBRIDGE
One daily rule — the only clock
It fires the dispatcher. Period.
`modules/dispatcher_lambda/`

→

DISPATCHER
Writes the cycle manifest
Starts one SFN per source
`src/lambdas/dispatcher/`

→

STEP FUNCTIONS
One per source · Maps its tables
Exec name = source + cycle_id
`modules/step-function/`

→

GLUE JOBS
source_download · bronze_pull
abap_transform · bronze_to_silver
`src/glue/glue_engine/jobs/`

2 · THE BARRIERS — A GATE BETWEEN EVERY PHASE

download_barrier
WAITS · every raw download
OPENS · the Bronze load (P2)
One failure → no Bronze at all
`download_barrier/`

→

transform_barrier
WAITS · every ABAP rebuild
OPENS · the Silver phase
Bad rebuild never reaches Silver
`transform_barrier/`

→

silver_barrier
WAITS · inputs from all sources
OPENS · the conform SFN
Cross-source joins need them all
`lambdas/silver_barrier/`

→

gold_barrier
WAITS · every expected table
OPENS · the dbt Gold build
Single-flight — one build a day
`lambdas/gold_barrier/`

3 · THE CONTROL PLANE — FOUR DYNAMODB TABLES, FOUR QUESTIONS

runs
What ran, and how did it go?
One row per table per stage
run_id + stage · 90-day TTL
`control_plane/runs.py`

watermarks
Where did we get to last time?
Last value pulled, per table
A failed run never advances it
`control_plane/watermarks.py`

pipeline-state
What is happening right now?
Running · alarms · freshness
Ops console polls every 10s
`control_plane/pipeline_state.py`

lineage_edges
What feeds what?
bronze → silver → gold → view
Deterministic id · re-emit safe
`control_plane/lineage_edges.py`

IN PLAIN ENGLISH

Barriers are await Promise.all() for data pipelines: nothing downstream starts until every upstream job has settled.

L21 · What Actually Runs It — Orchestration & the Control Plane

Slide: `_render/L21-orchestration-control-plane.html`

The point

One EventBridge rule fires each day. After that, **nothing is scheduled** — every subsequent phase is *unlocked* by a barrier Lambda that only fires when the phase before it is provably complete. That, plus a four-table control plane in DynamoDB, is the whole runtime.

Key ideas

- **The chain:** EventBridge (one daily rule) → dispatcher Lambda → one Step Functions execution per source → Glue jobs (`source_download` , `bronze_pull` , `abap_transform` , `bronze_to_silver`).
- **Barriers are `await Promise.all()` .** Each is the tail Task of a Step Function. It reads the cycle manifest, resolves every expected table to its latest run, and returns *waiting* unless all of them are terminal.
- **Four gates:** `download_barrier` (raw → Bronze), `transform_barrier` (ABAP rebuild → Silver), `silver_barrier` (per-source → cross-source conform), `gold_barrier` (Silver → dbt Gold).
- **All-or-nothing.** If any expected table failed, the barrier marks the cycle `*_skipped` and raises SNS. A partial raw landing must never become a partial Bronze.
- **Single-flight.** Every barrier claims a lock with a conditional DynamoDB write, so the *many* invocations that race to close a phase produce *one* decision — and one Gold build per day.
- **Idempotency is designed in, twice.** The dispatcher uses a deterministic Step Functions execution name (`<source>-<cycle_id>`), so a re-fire the same day is rejected outright; and Bronze loads MERGE on `merge_key` , so a replay overwrites instead of duplicating.
- **Retries are boring on purpose.** boto3 adaptive retry in every Lambda, Step Functions `Retry` / `Catch` around the barrier Task, and a `cycle_sweeper` on `rate(15 minutes)` that re-invokes a hung cycle.
- **The control plane answers four questions** — see the table below. Together they are the ops console's entire data source.

Words you'll hear

WORD	WHAT IT MEANS HERE
Cycle / `cycle_id`	One day's run of the whole pipeline, e.g. `2026-06-17` . Everything correlates on it
Manifest	The dispatcher's written promise: `expected[]` = the tables this cycle will produce
Barrier / gate	A Lambda that blocks the next phase until every table in the current phase is terminal
Terminal	A run reached `succeeded` , `failed` or `cancelled` — it is no longer in flight
Single-flight	Many callers race; a conditional write means exactly one wins and acts
Idempotent	Running it twice leaves the same result as running it once
Watermark	"How far did we get last time" — the resume pointer for an incremental pull
Lineage edge	A directed `upstream → downstream` fact, so you can answer "what feeds this?"

L21 · What Actually Runs It — Orchestration & the Control Plane

In this repo

- `src/lambdas/`
- `dispatcher/handler.py` — writes the manifest, starts one SFN per source
- `download_barrier/handler.py` — the P1→P2 gate; also counts chunked runs, not just latest ones
- `transform_barrier/handler.py` — the ABAP-rebuild gate
- `silver_barrier/handler.py` — the cross-source conform gate
- `gold_barrier/handler.py` — the dbt Gold gate; single-flight `gold_lock`
- `cycle_sweeper/handler.py` — the backstop that re-invokes a stalled cycle
- `src/shared/shared/control_plane/` — `runs.py`, `watermarks.py`, `pipeline_state.py`, `lineage_edges.py`, `coordination.py`
- `infra/modules/control_plane/main.tf` — the DynamoDB tables themselves
- `infra/modules/dispatcher_lambda/eventbridge.tf` — the one rule

The four tables, and what each answers

TABLE	QUESTION IT ANSWERS	KEY
`runs`	What ran, and how did it go?	`run_id` + `stage`; GSIs `by_table`, `by_cycle`; 90-day TTL
`watermarks`	Where did we get to last time?	`source_id` + `table_name`; a failed run never advances it
`pipeline-state`	What is happening right now?	`pipeline_id` = `<kind>:<id>` — pipeline / alarm / freshness / reconciliation
`lineage_edges`	What feeds what?	`edge_id` = `upstream|downstream|edge_type`, deterministic so re-emits overwrite

Do this

1. Read the docstring at the top of `gold_barrier/handler.py`. Write out its 4 steps in your own words.
2. In `download_barrier/handler.py`, find step **2b** (the chunk count). Explain why "latest run is terminal" is not enough for a chunked initial load.
3. In `dispatcher/handler.py`, find where the Step Functions execution name is built. Say what happens if the EventBridge rule fires twice in one day.
4. In `runs.py`, explain why a table's `bronze_pull` and `bronze_to_silver` are two rows and not one updated row.

You've got it when you can…

- Draw the daily cycle from cron to Gold, and put the four barriers on the right arrows.
- Explain a barrier to a backend developer in one sentence using `Promise.all()`, and then say where the analogy breaks (barriers are re-entrant and stateless; the state lives in DynamoDB).
- Say what happens to Gold when exactly one of 58 tables fails — and why that is the correct behaviour.
- Pick any of the four control-plane tables and say which operator question it answers, without looking.

MODULE 1 · LESSON 22

Same Architecture, New Client

The next engagement is apparel retail on Oracle. The skeleton is the one you just learned.

APPAREL GROUP · Enterprise Data & AI Platform on AWS · 12 weeks

Data Foundation | Price Optimization | Inter-store Transfer | Amazon Quick / Retail IQ

1 · WHAT STAYS THE SAME

THE SKELETON DOESN'T MOVE

- Raw first — land bytes untouched
- Medallion layers, same contracts
- Spec-driven YAML, one engine
- merge_key idempotency on load
- Barriers gate every hand-off
- dbt builds Gold + reporting views
- Control plane: same 4 DDB tables
- Terraform + OIDC CI/CD per env

`src/glue/specs/` · `src/glue/glue_engine/` · `src/dbt/` · `src/lambdas/` · `infra/`

2 · WHAT CHANGES

FIVE THINGS YOU RE-WRITE

- Connector: sap_hana → an Oracle JDBC source class
- Watermark column differs per source — no single FKDAT
- The MANDT client filter disappears — no SAP client
- 8 heterogeneous sources, incl. SaaS APIs — not just DBs
- Model becomes style / colour / size / season, not AAGM dept

`src/glue/glue_engine/sources/sap_hana.py` · `sources/__init__.py`

3 · THE EIGHT IN-SCOPE SOURCES

1 · Oracle Retail (RMS) — JDBC merchandising · cost · item + location	2 · Oracle SIM — JDBC store inventory positions	3 · Oracle XStore — JDBC in-store sales transactions	4 · Epsilon — SaaS API loyalty & customer master
5 · MoEngage — SaaS API campaign & engagement data	6 · Magento — DB / API e-commerce orders · customers · products	7 · Vemco Footfall — optional store footfall counts	8 · Irisys Footfall — optional store footfall counts

IN PLAIN ENGLISH

You already know 80% of the next project. The nouns change — Oracle, style, colour, season. The verbs don't.

L22 · Same Architecture, New Client — the Apparel Group bridge

Slide: `_render/L22-apparel-group-bridge.html`

The point

The next engagement is **Apparel Group — Enterprise Data & AI Platform on AWS**: 12 weeks, four workstreams (Data Foundation · Price Optimization · Inter-store Transfer · Amazon Quick / Retail IQ), eight in-scope sources. It is a different industry, a different vendor stack and a different dimensional model — and it is the *same skeleton you just spent Module 1 learning*. This lesson is the diff, not a new architecture.

Key ideas

- **What stays the same is the expensive part.** Medallion layers and their contracts, raw-first landing, the spec-driven YAML engine, `merge_key` idempotency, barriers between phases, dbt for Gold + reporting views, the four-table control plane, Terraform + OIDC CI/CD per environment.
- **What changes is the leaf nodes.** A new source connector class, different watermark columns, a different star schema — all of it plugs into unchanged machinery.
- **Connector:** `sap_hana` (HANA JDBC via `ngdbc.jar`) becomes an Oracle JDBC source class. It's a new file in `sources/` with an `@register("...")` decorator; the engine, the specs and the writers don't move.
- **Watermarks stop being uniform.** At Tamimi almost everything watermarks on a SAP date column such as `FKDAT`. Across eight heterogeneous sources you will have transaction timestamps, sequence numbers, API cursors and `updated_at` columns — one per source, declared per spec.
- **The MANDT client filter disappears.** `client: "100"` exists because SAP multiplexes clients into one table. Oracle Retail has no equivalent. Delete the concept; don't port it.
- **Not everything is a database.** Epsilon and MoEngage are SaaS APIs — paging, rate limits, tokens — so the download stage grows an API-shaped sibling to the JDBC path. Raw-first matters *more* here, because you often cannot re-request an old page.
- **The dimensional model becomes apparel retail:** style / colour / size / season replace AAGM department. Facts change grain (sales, inventory position, transfers, footfall, campaign response). Kimball still applies.
- **Two sources are optional.** Vemco and Irisys footfall are nice-to-have; design them as additive facts so the platform ships without them.
- **The bottom line: you already know 80% of the next project.** The nouns change; the verbs don't.

The eight in-scope sources

#	SOURCE	CARRIES	SHAPE
1	Oracle Retail (RMS)	merchandising, cost, item & location hierarchy	JDBC
2	Oracle SIM	store inventory positions	JDBC
3	Oracle XStore	in-store sales transactions	JDBC
4	Epsilon	loyalty & customer master	SaaS API
5	MoEngage	campaign & engagement data	SaaS API
6	Magento	e-commerce orders, customers, products	DB / API
7	Vemco Footfall	store footfall counts	optional
8	Irisys Footfall	store footfall counts	optional

L22 · Same Architecture, New Client — the Apparel Group bridge

Words you'll hear

WORD	WHAT IT MEANS HERE
RMS	Oracle Retail Merchandising System — the item/cost/hierarchy master
SIM	Oracle Store Inventory Management — what is on hand, per store
XStore	Oracle's POS — the transaction log at the till
SKU / style-colour-size	The apparel grain. A "style" fans out to colours, which fan out to sizes
Season	An apparel time dimension (SS26, AW26) that is *not* the calendar
Inter-store transfer	Moving stock between stores to fix a size-curve gap
Price optimization	Choosing markdown depth and timing from sell-through
MANDT	The SAP client column. Tamimi-only. It does not exist here
Footfall	People counted entering a store — the denominator for conversion rate

In this repo (the parts you will reuse verbatim)

- `src/glue/glue_engine/sources/` — `sap_hana.py`, `rds_jdbc.py`, `s3_landing` / `landed_files.py`, `excel_landing.py`, and `__init__.py` with the `@register` registry. **This is where the new Oracle class lands.**
- `src/glue/specs/` — `download/` (58), `bronze/` (67), `transform/` (14). Read `download/sap_zdsales.yaml` next to `bronze/sap_zdsales.yaml` to see `watermark_column`, `safety_buffer_days`, `jdbc_hash_partitions` and `merge_key` in one pair.
- `src/glue/glue_engine/spec.py` — the Pydantic contract every spec must satisfy
- `src/lambdas/` + `src/shared/shared/control_plane/` — barriers and control plane, unchanged
- `src/dbt/models/` — `staging/` → `marts/dims/` → `marts/gold/` → `marts/reporting/`; the shape survives, the models are rewritten
- `infra/modules/` + `bitbucket-pipelines.yml` — Terraform modules and the OIDC deploy per env

Do this

1. Take **Oracle Retail** `ITEM_MASTER` (or any table you know) and write the two spec YAMLs by hand: a `download/` spec (source type, watermark column, hash field, landing prefix, schema) and a `bronze/` spec (`merge_key`, same schema). Use `sap_zdsales.yaml` as the template.
2. Open `sources/sap_hana.py` and list every SAP-specific thing in it. That list is exactly your Oracle connector's to-do.
3. For each of the eight sources, write one line: what its watermark is, and what its natural key is. Where you can't answer, that's a discovery question for week 1.
4. Sketch the apparel star: name three facts and five dimensions.

You've got it when you can…

- Put any of the eight sources on the medallion map and say which existing file you would copy first.
- Name the four things that genuinely change (connector, watermark, MANDT gone, dimensional model) — and defend everything else as unchanged.
- Explain why raw-first is *more* important with SaaS APIs than with a database.
- Say the line and mean it: **you already know 80% of the next project.**